



**TRIBHUVAN UNIVERSITY  
INSTITUTE OF ENGINEERING  
THAPATHALI CAMPUS**

**A MINOR PROJECT REPORT  
ON  
DQN-BASED TRAFFIC SIGNAL CONTROL FOR ROAD TRAFFIC FLOW  
IMPROVEMENT AT THE THAPATHALI INTERSECTION**

**Submitted By:**

|                    |                |
|--------------------|----------------|
| Hemanta Bhusal     | (THA079BCT013) |
| Pratyoush Silwal   | (THA079BCT030) |
| Sachin Gautam      | (THA079BCT035) |
| Shoviyat Chapagain | (THA079BCT041) |

**Submitted To:**

Department of Electronics and Computer Engineering  
Thapathali Campus  
Kathmandu, Nepal

March 2026



**TRIBHUVAN UNIVERSITY  
INSTITUTE OF ENGINEERING  
THAPATHALI CAMPUS**

**A MINOR PROJECT REPORT  
ON  
DQN-BASED TRAFFIC SIGNAL CONTROL FOR ROAD TRAFFIC FLOW  
IMPROVEMENT AT THE THAPATHALI INTERSECTION**

**Submitted By:**

|                    |                |
|--------------------|----------------|
| Hemanta Bhusal     | (THA079BCT013) |
| Pratyoush Silwal   | (THA079BCT030) |
| Sachin Gautam      | (THA079BCT035) |
| Shoviyat Chapagain | (THA079BCT041) |

**Submitted To:**

Department of Electronics and Computer Engineering  
Thapathali Campus  
Kathmandu, Nepal

In partial fulfillment for the award of the  
Bachelors Degree in Electronics, Communication and Information Engineering

**Under the Supervision of**

Er. Saroj Sakhya

March 2026

## **CERTIFICATE OF APPROVAL**

The undersigned certify that they have read and recommended to the **Department of Electronics and Computer Engineering, IOE, Thapathali Campus**, a minor project work entitled **“DQN-Based Traffic Signal Control for Road Traffic Flow Improvement at the Thapathali Intersection”** submitted by **Hemanta Bhusal, Pratyoush Silwal, Sachin Gautam, Shoviyat Chapagain** in partial fulfillment for the award of Bachelor of Engineering in Electronics, Communication and Information Engineering. The project was carried out under the special supervision and within the time prescribed by the syllabus.

We found that the students to be hardworking, skilled and ready to undertake any related work to their field of study and hence we recommend the award of partial fulfillment of Bachelor’s Degree of Electronics, Communication, and Information Engineering.

---

Project Supervisor  
Er. Saroj Sakhya  
Department of Electronics and Computer  
Engineering  
Thapathali Campus, IOE

---

External Examiner

---

Project Coordinator  
  
Department of Electronics and Computer  
Engineering  
Thapathali Campus, IOE

---

Head of Department  
Er. Umesh Kanta Ghimire  
Department of Electronics and Computer  
Engineering  
Thapathali Campus, IOE

March 2026

## **DECLARATION**

We hereby declare that the project entitled, “**DQN-Based Traffic Signal Control for Road Traffic Flow Improvement at the Thapathali Intersection**” in the partial fulfillment of the requirements for the award of the Degree of Bachelor of Engineering **Electronics, Communication and Information Engineering** is a bona fide report of the work carried out by us. These materials contained within the report have not been submitted to any University or Institution for the award of any degree, and we are the only author of this complete work and no sources other than the listed ones have been used in this work.

|                    |                |
|--------------------|----------------|
| Hemanta Bhusal     | (THA079BCT013) |
| Pratyoush Silwal   | (THA079BCT030) |
| Sachin Gautam      | (THA079BCT035) |
| Shoviyat Chapagain | (THA079BCT041) |

**Date:** March 2026

## **COPYRIGHT**

The author has agreed that the library, Department of Electronics and Computer Engineering, Thapathali Campus, may make this report freely available for inspection. Moreover, the author has agreed that the permission for extensive copying of this project work for scholarly purpose may be granted by the professor/lecturer, who supervised the project work recorded herein or, in their absence, by the head of the department. It is understood that the recognition will be given to the author of this report and to the Department of Electronics and Computer Engineering, IOE, Thapathali Campus in any use of the material of this report. Copying of publication or other use of this report for financial gain without approval of the Department of Electronics and Computer Engineering, IOE, Thapathali Campus and author's written permission is prohibited.

Request for permission to copy or to make any use of the material in this project in whole or part should be addressed to department of Electronics and Computer Engineering, IOE, Thapathali Campus.

## ACKNOWLEDGEMENT

We would like to express our sincere gratitude to the Department of Electronics and Computer Engineering, Thapathali Campus, for providing the academic environment and resources necessary to initiate this project. We extend our heartfelt appreciation to our project supervisor, **Er. Saroj Sakhya** , for his invaluable guidance, consistent support, and constructive feedback throughout this stage.

We are also grateful to the traffic police personnel and concerned authorities for sharing practical insights regarding the existing traffic signal implementation and on-ground traffic management practices at the project location. Their input helped ensure that our proposed work remains aligned with real-world operational conditions. Finally, we would like to thank our teachers, classmates, and friends for their continuous encouragement and support.

Hemanta Bhusal            (THA079BCT013)

Pratyoush Silwal        (THA079BCT030)

Sachin Gautam          (THA079BCT035)

Shoviyat Chapagain    (THA079BCT041)

## ABSTRACT

Traffic congestion at the Thapathali intersection in Kathmandu causes significant delays, fuel waste, and commuter stress. Existing fixed-time signal control, reliant on pre-set cycle lengths and manual judgment, fails to adapt to the intersection's dynamic and asymmetric demand patterns, particularly during peak hours. This project proposes a Deep Reinforcement Learning (DRL)-based adaptive traffic signal control system using the Deep Q-Network (DQN) algorithm, trained within a SUMO microscopic traffic simulation environment constructed from OpenStreetMap data and calibrated against five days of field-collected Origin-Destination (OD) and vehicle count data from the Thapathali intersection. The DQN agent represents intersection state as a 21-dimensional vector encoding PCU-weighted vehicle loads on incoming and outgoing lanes, maximum waiting times at stoplines, and the currently active signal phase. It selects between two actions, STAY and NEXT, over a cyclic sequence of three signal phases at 15-second decision intervals, guided by a Max-Pressure based reward function that balances throughput and wait-time equity. The agent was trained over 300 episodes using experience replay and an epsilon-greedy exploration strategy. Comparative evaluation against a fixed 60-second signal plan on Day 5 traffic demand demonstrates consistent improvement across all 17 hours of operation: average waiting time was reduced by 22.90%, average queue length by 30.06%, and throughput efficiency (vehicles per wait-second) improved by 32.65%. The system performs best during low-to-moderate demand periods, where fixed-time control is structurally wasteful, and shows more modest gains during peak saturation where infrastructure capacity is the binding constraint. These results establish a credible simulation-based foundation for adaptive signal control at the Thapathali intersection, with identified pathways toward real-world deployment including multi-intersection coordination and integration of pedestrian and public transport flows.

**Keywords:** *Adaptive Control, Deep Q-Network (DQN), Deep Reinforcement Learning (DRL), Max-Pressure Theory, Queue Length, Simulation of Urban MObility (SUMO), Throughput, Traffic Signal Control*

## TABLE OF CONTENTS

|                                                                              |             |
|------------------------------------------------------------------------------|-------------|
| <b>CERTIFICATE OF APPROVAL . . . . .</b>                                     | <b>i</b>    |
| <b>DECLARATION . . . . .</b>                                                 | <b>ii</b>   |
| <b>COPYRIGHT . . . . .</b>                                                   | <b>iii</b>  |
| <b>ACKNOWLEDGEMENT . . . . .</b>                                             | <b>iv</b>   |
| <b>ABSTRACT . . . . .</b>                                                    | <b>v</b>    |
| <b>LIST OF FIGURES . . . . .</b>                                             | <b>ix</b>   |
| <b>LIST OF TABLES . . . . .</b>                                              | <b>x</b>    |
| <b>LIST OF ALGORITHMS . . . . .</b>                                          | <b>xi</b>   |
| <b>LIST OF ABBREVIATIONS . . . . .</b>                                       | <b>xii</b>  |
| <b>LIST OF SYMBOLS . . . . .</b>                                             | <b>xiii</b> |
| <br>                                                                         |             |
| <b>1. Introduction . . . . .</b>                                             | <b>1</b>    |
| 1.1. Background . . . . .                                                    | 1           |
| 1.2. Motivation . . . . .                                                    | 1           |
| 1.3. Problem Statement . . . . .                                             | 2           |
| 1.4. Objectives . . . . .                                                    | 2           |
| 1.5. Scope and Applications . . . . .                                        | 2           |
| <br>                                                                         |             |
| <b>2. Literature Review . . . . .</b>                                        | <b>4</b>    |
| 2.1. Conventional Traffic Signal Control Methods . . . . .                   | 4           |
| 2.2. RL for Traffic Signal Control . . . . .                                 | 4           |
| 2.3. DRL and Value-Based Control with DQN . . . . .                          | 5           |
| 2.4. Network-Level Coordination as Context for Future Extension . . . . .    | 5           |
| 2.5. Structured Policy Learning via Phase Competition and Symmetry . . . . . | 6           |
| 2.6. Simulation and Realistic Data Grounding . . . . .                       | 6           |
| 2.7. Summary and Project Positioning . . . . .                               | 6           |
| <br>                                                                         |             |
| <b>3. Requirements Analysis . . . . .</b>                                    | <b>8</b>    |
| 3.1. Functional Requirements . . . . .                                       | 8           |
| 3.2. Non-Functional Requirements . . . . .                                   | 8           |
| 3.3. System Requirements . . . . .                                           | 9           |



|                                                                                |           |
|--------------------------------------------------------------------------------|-----------|
| <b>4. System Architecture and Methodology . . . . .</b>                        | <b>10</b> |
| 4.1. System Architecture . . . . .                                             | 10        |
| 4.2. Use Case Diagram . . . . .                                                | 11        |
| 4.3. Class Diagram . . . . .                                                   | 12        |
| 4.4. Sequence Diagram . . . . .                                                | 14        |
| 4.5. Data Analysis . . . . .                                                   | 15        |
| 4.6. Environment Setup for Traffic Simulation . . . . .                        | 17        |
| 4.7. DQN Agent Development . . . . .                                           | 19        |
| 4.8. Design Justification . . . . .                                            | 20        |
| 4.9. Hyperparameter Selection and Sensitivity . . . . .                        | 22        |
| 4.10. Training Process . . . . .                                               | 26        |
| 4.11. Algorithms Used . . . . .                                                | 26        |
| <b>5. Implementation Details . . . . .</b>                                     | <b>29</b> |
| 5.1. Vehicle Flow Generation . . . . .                                         | 29        |
| 5.2. Traffic Simulation Environment (SUMO) . . . . .                           | 32        |
| 5.3. Environment Wrapper Implementation . . . . .                              | 34        |
| 5.4. Traffic Flow and Signal Phase Design at Thapathali Intersection . . . . . | 35        |
| 5.5. DQN Agent Implementation . . . . .                                        | 39        |
| 5.6. Training Pipeline Implementation . . . . .                                | 39        |
| 5.7. Configuration Management . . . . .                                        | 40        |
| 5.8. Metric Calculation . . . . .                                              | 40        |
| <b>6. Results and Analysis . . . . .</b>                                       | <b>42</b> |
| 6.1. Simulation Environment Configuration . . . . .                            | 42        |
| 6.2. Training Convergence and Learning Trajectory . . . . .                    | 42        |
| 6.3. Wait Time Reduction Analysis . . . . .                                    | 43        |
| 6.4. Queue Length Management Effectiveness . . . . .                           | 44        |
| 6.5. Throughput Efficiency Analysis . . . . .                                  | 45        |
| 6.6. Consolidated Performance Comparison . . . . .                             | 46        |
| 6.7. Overall Performance Summary and Analysis . . . . .                        | 47        |
| <b>7. Conclusion . . . . .</b>                                                 | <b>51</b> |
| 7.1. Achievements . . . . .                                                    | 51        |
| 7.2. Limitations . . . . .                                                     | 51        |
| 7.3. Future Work . . . . .                                                     | 51        |
| 7.4. Final Remarks . . . . .                                                   | 52        |
| <b>APPENDICES . . . . .</b>                                                    | <b>54</b> |

|                               |           |
|-------------------------------|-----------|
| <b>A. Code Snippets</b>       | <b>54</b> |
| A.1. DQN Network Architecture | 54        |
| A.2. Replay Buffer            | 54        |
| A.3. DQN Agent                | 55        |
| A.4. Model Training           | 56        |
| A.5. Model Saving and Loading | 56        |
| A.6. Summary                  | 57        |
| <b>B. Project Budget</b>      | <b>58</b> |
| <b>C. Project Timeline</b>    | <b>59</b> |
| C.1. Gantt Chart              | 59        |
| <b>D. Summary of Data</b>     | <b>60</b> |
| <b>REFERENCES</b>             | <b>63</b> |

## LIST OF FIGURES

|                                                                                                           |    |
|-----------------------------------------------------------------------------------------------------------|----|
| Figure 4-1. Block Diagram of DQN-Based Traffic Signal Control System . . . .                              | 11 |
| Figure 4-2. Use case diagram of the Thapathali DQN traffic signal control system                          | 12 |
| Figure 4-3. Class diagram of the Thapathali DQN traffic signal control system .                           | 13 |
| Figure 4-4. Sequence diagram of the Thapathali DQN traffic signal control system                          | 15 |
| Figure 4-5. SUMO Environment Block Diagram . . . . .                                                      | 18 |
| Figure 4-6. Neural Network Architecture for Q-value Prediction . . . . .                                  | 21 |
| Figure 5-1. Thapathali Road Traffic flow and Traffic light control stage<br>representation . . . . .      | 35 |
| Figure 5-2. Implemented Traffic Signal Phases for fixed-timer and DQN-based<br>Agent . . . . .            | 37 |
| Figure 6-1. SUMO Simulation Environment of Thapathali Intersection . . . . .                              | 42 |
| Figure 6-2. Episode Reward and Epsilon Decay over 300 Training Episodes . . .                             | 43 |
| Figure 6-3. Day 5 Average Wait Time by Hour: Fixed 60s vs DQN (Episode 300)                               | 44 |
| Figure 6-4. Day 5 Average Total Queue Length by Hour: Fixed 60s vs DQN<br>(Episode 300) . . . . .         | 45 |
| Figure 6-5. Efficiency Ratio of Throughput per Second of Wait Time: F60 vs<br>DQN (Episode 300) . . . . . | 46 |
| Figure C-1. Project Gantt Chart . . . . .                                                                 | 59 |

## LIST OF TABLES

|                                                                                           |    |
|-------------------------------------------------------------------------------------------|----|
| Table 2.7.1. Comparison of Traffic Signal Control Methods . . . . .                       | 7  |
| Table 3.1.1. Functional Requirements for the Project . . . . .                            | 8  |
| Table 3.2.1. Non-Functional Requirements for the Project . . . . .                        | 9  |
| Table 4.5.1. Vehicle Class and Adopted PCU Factors . . . . .                              | 16 |
| Table 5.2.1. Vehicle Type Configurations in SUMO Simulation . . . . .                     | 33 |
| Table 6.6.1. Hour-by-Hour Performance: Wait Time and Queue Length (Episode 300) . . . . . | 46 |
| Table 6.6.2. Hour-by-Hour Performance: Throughput Efficiency (Episode 300) .              | 47 |
| Table B.0.1. Project Budget Allocation . . . . .                                          | 58 |
| Table D.0.1. 15-Minute Interval Vehicle Counts by Class: Day 1 (06:00–15:15) .            | 60 |
| Table D.0.2. 15-Minute Interval Vehicle Counts by Class: Day 1 (15:15–22:15) .            | 61 |
| Table D.0.3. PCU OD Data: Days 1 & 2 . . . . .                                            | 61 |
| Table D.0.4. PCU OD Data: Days 3 & 4 . . . . .                                            | 62 |
| Table D.0.5. PCU OD Data: Day 5 . . . . .                                                 | 62 |

## LIST OF ALGORITHMS

|                                                                                    |    |
|------------------------------------------------------------------------------------|----|
| 4.11.1. Reward Calculation . . . . .                                               | 27 |
| 4.11.2. DQN Training Process . . . . .                                             | 28 |
| 5.1.1. Perfect Vehicle Allocation with Remainder Tracking . . . . .                | 29 |
| 5.1.2. Batch Perfect Allocation using piecewise linear interpolation . . . . .     | 30 |
| 5.1.3. Mathematical Formulation of largest remainder method for vehicle allocation | 31 |

## **LIST OF ABBREVIATIONS**

|       |                                   |
|-------|-----------------------------------|
| API   | Application Programming Interface |
| DQN   | Deep Q-Network                    |
| DRL   | Deep Reinforcement Learning       |
| ML    | Machine Learning                  |
| MSE   | Mean Squared Error                |
| OD    | Origin-Destination                |
| OSM   | OpenStreetMap                     |
| PCU   | Passenger Car Unit                |
| PEP8  | Python Enhancement Proposal 8     |
| ReLU  | Rectified Linear Unit             |
| RL    | Reinforcement Learning            |
| SUMO  | Simulation of Urban MObility      |
| TRACI | Traffic Control Interface         |
| XML   | eXtensible Markup Language        |

## LIST OF SYMBOLS

|                         |                                                  |
|-------------------------|--------------------------------------------------|
| $a_n$                   | Allocated vehicle count for OD pair $n$          |
| $a'$                    | Next action in Bellman update                    |
| $\alpha$                | Learning rate for Adam optimiser                 |
| $\epsilon$              | Exploration rate in $\epsilon$ -greedy strategy  |
| $\epsilon_{decay}$      | Epsilon decay factor                             |
| $\epsilon_{min}$        | Minimum exploration rate                         |
| $f_n$                   | Fractional part of allocation for OD pair $n$    |
| $\lfloor \cdot \rfloor$ | Floor function for integer allocation            |
| $\gamma$                | Discount factor for future rewards               |
| $L_{out}$               | Set of outgoing lanes                            |
| $L_{stop}$              | Set of stopline lanes                            |
| $\lambda$               | Reward weighting factor for wait time penalty    |
| $\mathcal{L}$           | MSE loss between target and current Q-values     |
| $p_1 \dots p_7$         | PCU-weighted load on origin lanes                |
| $p_n$                   | Proportion for OD pair $n$ in vehicle allocation |
| $p_{o1} \dots p_{o5}$   | PCU-weighted load on outgoing lanes              |
| $PCU(l)$                | Total PCU-weighted vehicle load on lane $l$      |
| $PCU(v)$                | PCU value of individual vehicle $v$              |
| $Q_{current}$           | Current Q-value from Q-network                   |
| $Q(s, a)$               | Q-value for state $s$ and action $a$             |
| $Q_{target}$            | Target Q-value from Bellman equation             |
| $r_n^{new}$             | New remainder for OD pair $n$                    |
| $r_n^{prev}$            | Previous remainder for OD pair $n$               |
| $R$                     | Remainder count in largest remainder method      |
| $R$                     | Reward value at current decision step            |
| $r$                     | Immediate reward received                        |

|                 |                                         |
|-----------------|-----------------------------------------|
| $s'$            | Next state after action execution       |
| $s_t$           | State vector at decision step $t$       |
| $\Sigma$        | Summation operator                      |
| $v_c$           | Total vehicle count for class $c$       |
| $w^{acc}(v)$    | Accumulated waiting time of vehicle $v$ |
| $w_1 \dots w_6$ | Maximum waiting time on stopline lanes  |



# **1. INTRODUCTION**

## **1.1. Background**

In urban cities like Kathmandu, traffic congestion is a growing problem, causing wasted time, pollution, and economic loss. Currently, in Kathmandu, traffic is monitored through cameras, and signal timings are set manually based on these observations or managed manually by traffic police. However, these methods are not very effective; instead of reducing congestion, they often cause additional delays because this system often fails during sudden surges, accidents, or unusual traffic patterns. This limitation highlights the need for more intelligent, adaptive traffic management systems. Since most delays originate at major junctions, this project focuses specifically on improving intersection-level signal control at the Thapathali intersection rather than addressing city-wide traffic management.

## **1.2. Motivation**

The daily commute in Kathmandu, particularly around busy intersections like Thapathali, has long been a source of frustration for its residents. Traffic congestion is a persistent issue, especially as the city continues to grow and traffic patterns become increasingly unpredictable. Conventional traffic control methods, which rely on fixed patterns and limited flexibility, are proving insufficient to address the dynamic and often chaotic nature of Kathmandu's roadways. As the city's population expands and the number of vehicles on the road increases, the need for a more intelligent and adaptive approach to traffic signal control becomes critical.

This project seeks to tackle the limitations of traditional traffic management systems by employing the power of DRL. The aim is to develop an adaptive traffic signal control framework that can dynamically respond to real-time traffic conditions, improving traffic efficiency, reducing congestion, and making daily commutes smoother for the people of Kathmandu, especially around congested areas like Thapathali.

To test and validate this approach, we turn to advanced simulation tools like SUMO, which offers a scalable and safe platform for replicating real-world traffic scenarios. With accurate road network data from OpenStreetMap (OSM), SUMO allows us to simulate the conditions in Kathmandu and evaluate AI-based traffic control strategies. This simulation environment provides an ideal space to refine the system before real-world implementation.

### **1.3. Problem Statement**

Current Thapathali intersection traffic signal systems are largely static or semi-adaptive, failing to respond effectively to complex and fluctuating traffic patterns. This leads to increased vehicle waiting times and travel delays, higher fuel consumption and air pollution, and inefficient coordination across intersections. The problem this project addresses is how to design a traffic signal control system that can adapt in real-time to dynamic traffic conditions, using a learning-based approach to optimize flow, minimize delays, and improve overall intersection throughput.

A DQN-based traffic signal controller is expected to reduce average waiting time and queue length compared to fixed-time control under realistic Thapathali traffic demand because it can continuously adjust phase selection and green allocation based on real-time traffic states (e.g., queue lengths, waiting times, current phase, and elapsed duration). Unlike fixed-time plans that rely on pre-set splits and cannot respond to demand fluctuations, asymmetric approach volumes, or sudden surges, the DQN agent can reduce wasted green time on low-demand approaches, prevent queue buildup and spillback, and improve throughput under the same demand conditions.

### **1.4. Objectives**

The primary objectives of this project are:

- To develop a DQN-based DRL agent for adaptive traffic signal control using SUMO in a realistic simulation of the Thapathali Intersection.
- To compare the performance of the proposed agent against conventional traffic signal control methods.

### **1.5. Scope and Applications**

This project focuses on developing a DQN-based DRL agent for adaptive traffic signal control using realistic simulation of the Thapathali intersection. The scope defines what will be covered in the project and sets clear boundaries for its implementation. The intended applications include simulation-based testing of signal timing strategies for a Thapathali intersection, comparative performance analysis against baseline signal plans, and decision support for traffic engineering studies prior to real-world deployment. The proposed system is designed to be scalable; subject to time and data availability, it can be extended to multi-intersection coordination in future work.

### **1.5.1. Scope**

The scope of the Project encompasses the objectives, boundaries, and key focus areas of the work.

- To configure the Thapathali intersection map into SUMO to simulate realistic traffic patterns.
- To design the DQN-based DRL agent with state representations, reward functions, and action spaces.
- To evaluate model performance using metrics such as average waiting time, queue lengths, and vehicle throughput.

## **2. LITERATURE REVIEW**

Urban intersections are recurring bottlenecks where conflicting traffic streams compete for limited green time. Conventional signal timing can improve intersection performance, but it often depends on simplified assumptions and periodic manual retiming. In practice, traffic demand varies by time of day and day type, and static plans may not remain effective under changing conditions. For the Thapathali Intersection specifically, prior local work has evaluated the intersection as an isolated study area using observed peak-hour traffic volumes and operational performance measures (e.g., delay, queue-related indicators, and level of service), providing realistic context and baseline conditions for simulation-based signal studies [1].

### **2.1. Conventional Traffic Signal Control Methods**

Traditional signal control approaches for an isolated intersection commonly produce a cycle-based plan consisting of a phase sequence, cycle length, and green splits. A widely used classical method is Webster-type timing, which computes timing parameters from traffic volume and related assumptions to generate a fixed-time plan for an individual intersection [2]. Beyond fixed-time plans, actuated and rule-based controllers adjust phase changes based on detector inputs and predefined rules, offering limited responsiveness without learning [2]. At a higher level, transportation methods can be grouped into (i) fixed-time control, (ii) actuated/rule-based control, (iii) selection-based adaptive control (choosing among precomputed plans), and (iv) optimization-based adaptive control that explicitly optimizes an objective under simplified flow models [2]. These approaches are practical and interpretable, but they can require careful calibration, may rely on oversimplified traffic representations, and may not adapt effectively to complex, time-varying patterns.

A particularly relevant conventional baseline is max-pressure control, which uses real-time queue information to select phases that balance upstream and downstream congestion [2]. Max-pressure has theoretical properties related to throughput under certain conditions, making it a strong comparator for learning-based controllers. However, it remains a greedy decision rule and does not learn long-term policies from interaction.

### **2.2. RL for Traffic Signal Control**

Reinforcement Learning (RL) reframes signal control as a sequential decision-making process: an agent observes traffic conditions, takes an action (e.g., choosing the next phase or adjusting green time), and receives a reward reflecting operational quality (e.g., reduced delay or queues). RL is attractive for traffic signal control because “optimal”

timing is difficult to label in advance for all scenarios; instead, policies can be learned through trial-and-error interaction in a simulator [2]. For a single intersection, the main design choices typically involve: (1) the state representation (what the agent observes), (2) the action space (what the agent can control), and (3) the reward function (what the agent is incentivized to optimize). Practical state elements often include per-lane vehicle counts/queues and the current signal phase, while rewards are usually derived from delay/queue metrics or throughput-related measures [2, 3].

### **2.3. DRL and Value-Based Control with DQN**

DRL extends RL by using neural networks to approximate value functions or policies, enabling learning from richer traffic states. A common value-based method is the DQN, which estimates action-values and can be trained using experience replay to stabilize learning [4]. A key challenge in many DRL traffic-signal studies is that performance can be highly sensitive to reward and state design; overly complex states and heuristic rewards can slow training or lead to unstable outcomes. To address this, one approach explicitly grounds DRL design in transportation theory by connecting RL to max-pressure concepts: the reward is shaped to align with pressure-based objectives, and the state is designed to retain only the information necessary to optimize that objective [4]. This is important even for a single-intersection project because principled reward/state choices can improve learning stability and reduce unnecessary modeling complexity.

In addition to reward/state design, evaluation practice in DRL traffic signal control typically compares learned policies to conventional baselines such as fixed-time plans and other transportation-inspired controllers, using metrics like travel time, delay, queue length, and throughput to quantify benefits [2, 4]. For single-intersection studies, these comparisons are crucial to demonstrate that learning provides value beyond strong non-learning baselines.

### **2.4. Network-Level Coordination as Context for Future Extension**

Although the present project targets one intersection, the literature on multi-intersection control highlights why coordination is beneficial and how learning methods represent neighboring influence. One DRL approach models traffic signal control as a Markov game where each intersection is controlled by an agent, and neighboring intersections influence one another through learned cooperation [3]. In that formulation, an agent's observation includes its current phase (often as a one-hot vector) and lane-level vehicle counts, while a learned attention mechanism assigns different weights to neighbors depending on spatial and temporal conditions [3]. The method's Q-learning style objective reflects value-based learning with a function approximator and a loss that penalizes Q-value prediction errors across intersections and time steps [3]. Even when

applied to a single intersection, these ideas motivate using lane-level counts and explicit phase information as core state variables and reinforce the importance of scalable representations if later extension to multi-intersection coordination is desired.

## **2.5. Structured Policy Learning via Phase Competition and Symmetry**

Another relevant direction improves learning efficiency by structuring how actions (phases) are represented. A phase-competition approach argues that many traffic states are equivalent under rotation and flipping (symmetry) and that RL agents should generalize across such cases without relearning from scratch [5]. The method leverages this observation by modeling competition among phases, prioritizing movements with higher demand when conflicts exist, and explicitly designing the model to be invariant to symmetric transformations [5]. For a single intersection, this supports two practical design lessons: (1) the action space should reflect meaningful phase choices rather than overly granular controls that increase exploration burden, and (2) the model/state design can exploit structural regularities in intersection layouts to improve sample efficiency.

## **2.6. Simulation and Realistic Data Grounding**

DRL traffic signal control is commonly developed and validated in simulation for safety, repeatability, and fast iteration. SUMO is frequently used as it can execute signal actions, simulate vehicle movements, and provide state measurements required by controllers [5]. For Thapathali, local operational data provides a valuable foundation for building realistic simulation demand and for defining baseline performance expectations. The Thapathali case study reports peak-hour traffic volumes (in Passenger Car Unit (PCU)) for multiple observation days and provides approach-level volume distributions, which can be mapped into simulation flows [1]. It also reports performance statistics such as degree of saturation, average delay, level of service, and queue-related measures for the existing intersection operation and for candidate improvement options [1]. Such data is particularly useful for a minor project because it enables (i) calibration of traffic demand patterns in simulation, and (ii) evaluation of whether learned signal policies improve meaningful operational indicators under realistic conditions.

## **2.7. Summary and Project Positioning**

Overall, the literature indicates a progression from fixed-time and rule-based controllers toward learning-based adaptive control capable of reacting to dynamic traffic states. Conventional transportation methods remain important as baselines and as sources of theory-driven guidance (e.g., max-pressure). DRL methods such as DQN provide a practical foundation for adaptive control, but their success depends strongly on state, action, and reward design. Theory-grounded DRL designs linked to established

traffic principles can improve stability and interpretability [4], while attention-based cooperation and symmetry-aware phase modeling illustrate pathways for scalability and efficiency [3, 5]. For a single-intersection implementation at Thapathali, a DQN-based DRL agent trained in SUMO and grounded in realistic local traffic volumes provides a manageable scope while still aligning with the most effective practices observed in the literature [1, 2].

Table 2.7.1. Comparison of Traffic Signal Control Methods

| Method        | Control Logic                                                                                  | Adaptivity                                                                       | Data Needs                                                               |
|---------------|------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| Fixed Time    | Pre-set cycle length, phase order, and green splits (static plan).                             | Low: no real-time response; only changes if manually retimed.                    | Historical/observed volumes for timing design; no training required.     |
| Max Pressure  | Selects phase with highest pressure using upstream vs downstream queue imbalance (rule-based). | High: reacts instantly to queues, but remains a fixed greedy rule (no learning). | Real-time queue/occupancy (ideally downstream too); no training dataset. |
| DQN-based DRL | Learns a signal policy by maximizing reward through simulation interaction.                    | High: learns and adapts a policy to objectives and constraints.                  | Training episodes in SUMO; state features; compute and tuning effort.    |

### 3. REQUIREMENTS ANALYSIS

This chapter presents a comprehensive analysis of the requirements governing the DQN-based adaptive traffic signal control system. It covers functional, non-functional, hardware, and software requirements that collectively define the expected behavior, performance constraints, and operational environment necessary for successfully simulating and optimizing traffic flow at the Thapathali intersection using SUMO and reinforcement learning, with the aim of outperforming conventional fixed-timer signal control plans.

#### 3.1. Functional Requirements

The functional requirements define the core capabilities the system must deliver, covering simulation environment setup, DQN agent implementation, real-time traffic state processing, signal control via TraCI, performance metric logging, and baseline comparison between the DQN agent and fixed timer control.

Table 3.1.1. Functional Requirements for the Project

| ID     | Description                                                                                                                          |
|--------|--------------------------------------------------------------------------------------------------------------------------------------|
| FR-001 | The system shall import the Thapathali intersection map from OpenStreetMap and configure it in SUMO with realistic traffic patterns. |
| FR-002 | The system shall implement a DQN-based agent capable of changing traffic signal phases to optimize traffic flow.                     |
| FR-003 | The system shall update the DQN target network periodically to facilitate stable real-time learning.                                 |
| FR-004 | The system shall accept traffic state inputs including vehicle counts, queue lengths, and waiting times.                             |
| FR-005 | The system shall output traffic light control commands via Traffic Control Interface (TRACI) based on the DQN agent's decisions.     |
| FR-006 | The system shall log key performance metrics (e.g., queue length, throughput) for post-simulation analysis.                          |
| FR-007 | The system shall visualize simulation progress via SUMO-GUI for monitoring purposes.                                                 |
| FR-008 | The system shall allow comparative evaluation against baseline fixed-time or actuated signal plans.                                  |

#### 3.2. Non-Functional Requirements

Non-functional requirements covering performance, reliability, usability, and portability.



Table 3.2.1. Non-Functional Requirements for the Project

| ID      | Category        | Description                                                                                                                       |
|---------|-----------------|-----------------------------------------------------------------------------------------------------------------------------------|
| NFR-001 | Performance     | The DQN agent shall compute and apply a new traffic signal phase within 1 second per simulation step.                             |
| NFR-002 | Performance     | The system shall achieve at least 15–20% reduction in average waiting time and queue length compared to the baseline signal plan. |
| NFR-003 | Performance     | The system shall achieve at least 10% increase in vehicle throughput compared to the baseline.                                    |
| NFR-004 | Portability     | The system shall be fully operational on Windows 10+ and Ubuntu 20.04+.                                                           |
| NFR-005 | Usability       | The system shall provide real-time visualization options (SUMO-GUI) and clear performance logs.                                   |
| NFR-006 | Maintainability | The code shall be modular, well-documented, and follow Python Enhancement Proposal 8 (PEP8) conventions.                          |
| NFR-007 | Security        | No personal or sensitive data is processed; only aggregated traffic metrics are used.                                             |

### 3.3. System Requirements

The system requirements outline the minimum and recommended hardware specifications necessary to run the SUMO simulation and train the DQN agent, along with the required software dependencies including the SUMO simulation platform, Python environment, and key libraries for deep learning, data processing, and visualisation.

#### 3.3.1. Hardware Requirements

- **Minimum:** 64-bit CPU, 8 GB RAM, 10 GB free disk space.
- **Recommended:** 4+ core CPU, 16 GB RAM, SSD storage.
- **Optional (Training):** Dedicated GPU with 4+ GB VRAM or Google Colab access.

#### 3.3.2. Software Requirements

- **Operating System:** Windows 10+, MacOS or Ubuntu 20.04+.
- **SUMO:** Eclipse SUMO 1.26.0 or above.
- **Python:** 3.8 or higher.
- **Key Libraries:** TRACI, pandas, PyTorch, NumPy, Matplotlib.
- **Development Tools:** VS Code, Git, and GitHub.

## 4. SYSTEM ARCHITECTURE AND METHODOLOGY

### 4.1. System Architecture

The system employs a Deep Q-Network architecture for adaptive traffic signal control at the Thapathali T1 intersection, integrating the SUMO microscopic traffic simulation platform as the environment with a custom reinforcement learning framework designed specifically for the intersection's geometry and traffic demand characteristics. The overall architecture follows the standard reinforcement learning loop where the agent observes the current state of the intersection, selects a signal control action, receives a reward signal reflecting the quality of that decision, and updates its policy through experience-driven learning.

The state representation module processes real-time traffic data collected at each 15-second decision step through SUMO's TraCI interface. PCU-weighted vehicle loads are computed for the 7 origin lanes approaching the intersection and the 5 outgoing lanes discharging traffic away from it, capturing the instantaneous flow conditions across all major approaches. Maximum waiting times are extracted for each of the 6 stopline lanes, identifying the most severely delayed vehicles queuing at the signal. The currently active signal phase is encoded as a 3-dimensional one-hot vector to inform the agent of its current operating context. Together these components are concatenated into a fixed-length 21-dimensional feature vector that comprehensively represents the intersection status at each decision point.

This state vector feeds into dual neural networks that form the core of the DQN learning mechanism. The Q-network receives the 21-dimensional state vector as input and passes it through three fully connected hidden layers of 128, 64, and 32 neurons respectively using ReLU activation functions, designed to capture complex non-linear relationships between traffic states and optimal signal timing decisions. The output layer produces 2 Q-values, one for each possible action, representing the agent's estimated cumulative future reward for selecting each action from the current state. A separate target network maintains an identical architecture but updates its weights less frequently by periodically copying from the Q-network, providing stable learning targets that prevent oscillations and divergence during training.

A replay buffer stores up to 10,000 past experience tuples, each consisting of the observed state, selected action, received reward, resulting next state, and episode termination flag. During each training step, a random mini-batch of 64 transitions is sampled from this buffer to compute the Q-learning update, breaking the temporal correlations between consecutive experiences and preventing the network from overfitting to recent

transitions. This experience replay mechanism significantly improves the stability and sample efficiency of the learning process across the full training run of 300 episodes.

An  $\epsilon$ -greedy exploration strategy governs action selection throughout training. In early episodes the agent selects actions randomly with high probability, broadly exploring the state space and populating the replay buffer with diverse experiences. As training progresses  $\epsilon$  decays multiplicatively after each episode, gradually shifting the agent towards exploiting its learned Q-values for action selection. By the final episodes the agent operates almost entirely on its learned policy, with only a small residual probability of random exploration retained to avoid becoming trapped in suboptimal behaviours.

The action space consists of 2 discrete actions - STAY and NEXT - applied over a cyclic sequence of 3 predefined signal phases. At each 15-second decision interval the agent either maintains the current active phase to continue serving the same traffic direction or advances to the next phase in the cycle to serve a different approach. Each phase transition includes a 3-second yellow interval to ensure safe and real

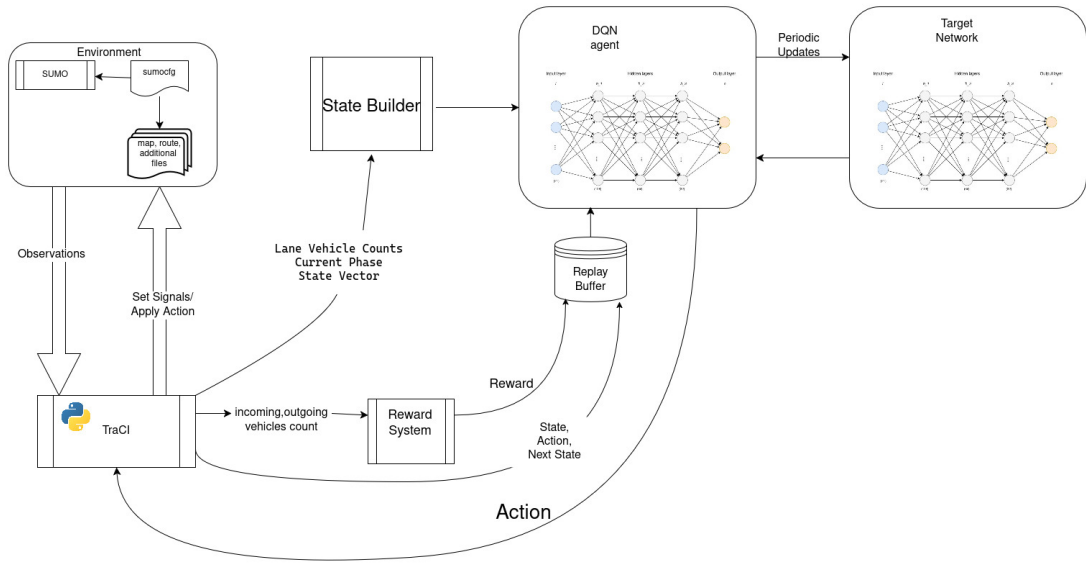


Figure 4-1. Block Diagram of DQN-Based Traffic Signal Control System

## 4.2. Use Case Diagram

Figure 4-2 presents the use case diagram for the Thapathali DQN Traffic Signal Control System. The system involves four actors: the Traffic Engineer, Evaluation Analyst, SUMO Simulator, and DQN Agent. The Traffic Engineer initiates the system by configuring the simulation, loading the trained model, and starting the simulation. The Evaluation Analyst is responsible for generating performance metrics and triggering the comparison between the DQN agent and the fixed timer baseline. The SUMO Simulator acts as the external simulation environment supporting both training and evaluation.

The DQN Agent autonomously observes traffic state, selects signal actions, stores experiences, applies signal phases, and calculates rewards during training. The Train DQN Model use case includes five mandatory sub-processes: observe traffic state, select signal action, apply signal phase, store experience, and calculate reward, all executed at every decision step. Update Standalone Signals extends the training process under specific traffic conditions, while Compare DQN vs Fixed Timer extends from Generate Performance Metrics and triggers the Run Test Scenario use case for evaluation.

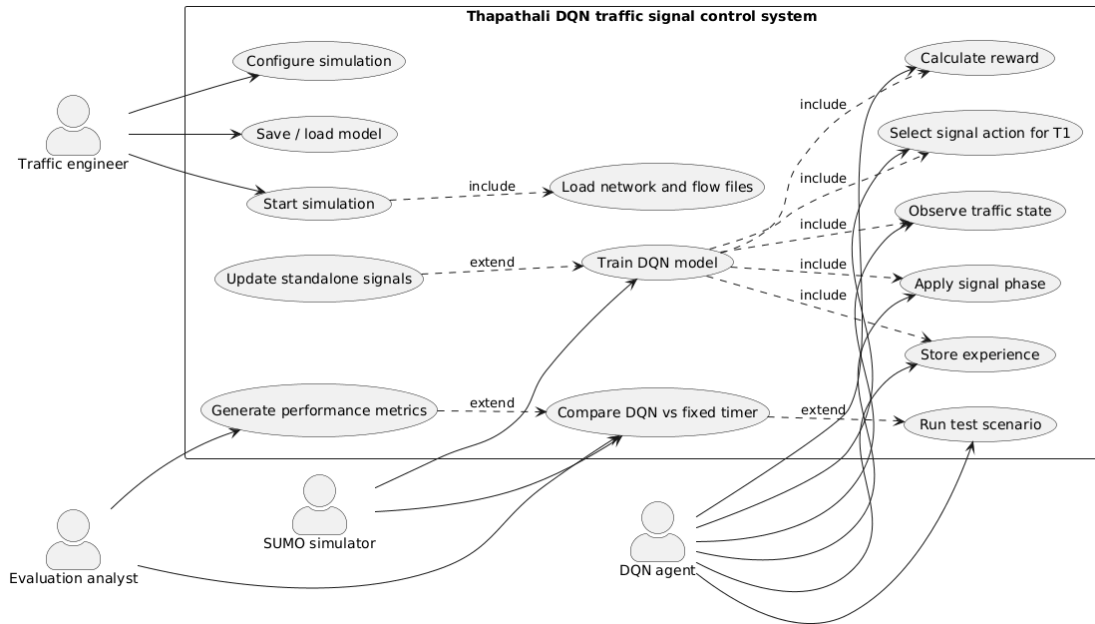


Figure 4-2. Use case diagram of the Thapathali DQN traffic signal control system

### 4.3. Class Diagram

Figure 4-3 presents the class diagram illustrating the structural relationships between the five core classes of the system. The TrainDQN class serves as the entry point, creating the SumoEnvironment and using the DQNAgent to manage the training loop. The SumoEnvironment class encapsulates all simulation interaction through SUMO's TraCI interface, maintaining lane-level traffic data and exposing methods for state observation, reward calculation, phase control, and metric collection. The DQNAgent class manages the reinforcement learning logic, holding references to two DQNetwork instances – the Q-network for action selection and the target network for stable learning targets – and a ReplayBuffer for experience storage. The DQNetwork class implements the neural network with three fully connected hidden layers. The config class acts as a shared configuration module, read by both SumoEnvironment and DQNAgent to maintain consistent hyperparameters and simulation settings throughout

the system. The FixedTimerController class operates independently and compares against SumoEnvironment during evaluation.

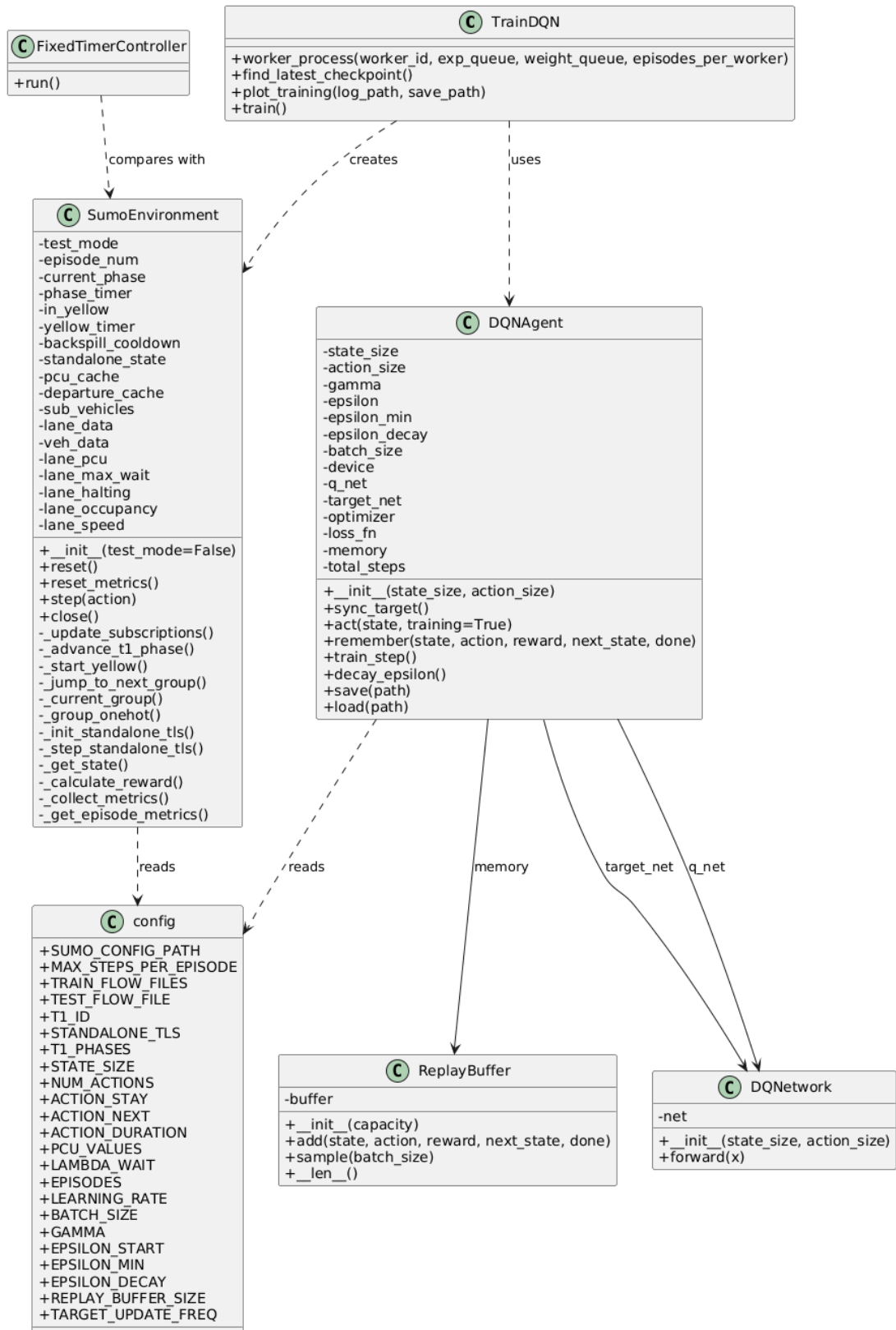


Figure 4-3. Class diagram of the Thapathali DQN traffic signal control system

#### 4.4. Sequence Diagram

Figure 4-4 presents the sequence diagram illustrating the end-to-end flow of the system from initialisation through training to final evaluation. The traffic engineer triggers the process by calling `train()`, which initialises the `DQNAgent` with its neural networks and replay buffer and the `SumoEnvironment`, which loads the network and flow files into SUMO. During the 300-episode training loop, each episode begins with a reset and proceeds through repeated decision steps where the agent observes the state, selects an action, and applies the signal phase for 15 seconds of green time. When the agent selects NEXT, a 3-second yellow transition is executed before switching to the next phase, ensuring safe and realistic signal operation. The agent then receives the reward, stores the experience, and performs a training step. After each episode epsilon decays, the target network synchronises every 10 episodes, and checkpoints are saved every 5 episodes. Upon completion the final model is saved and training plots are generated. During evaluation the Evaluation Analyst runs the `FixedTimerController` on the same test scenario to collect baseline metrics, then runs the DQN agent in test mode using the saved model to collect DQN metrics, and finally compares both results to generate the performance report.

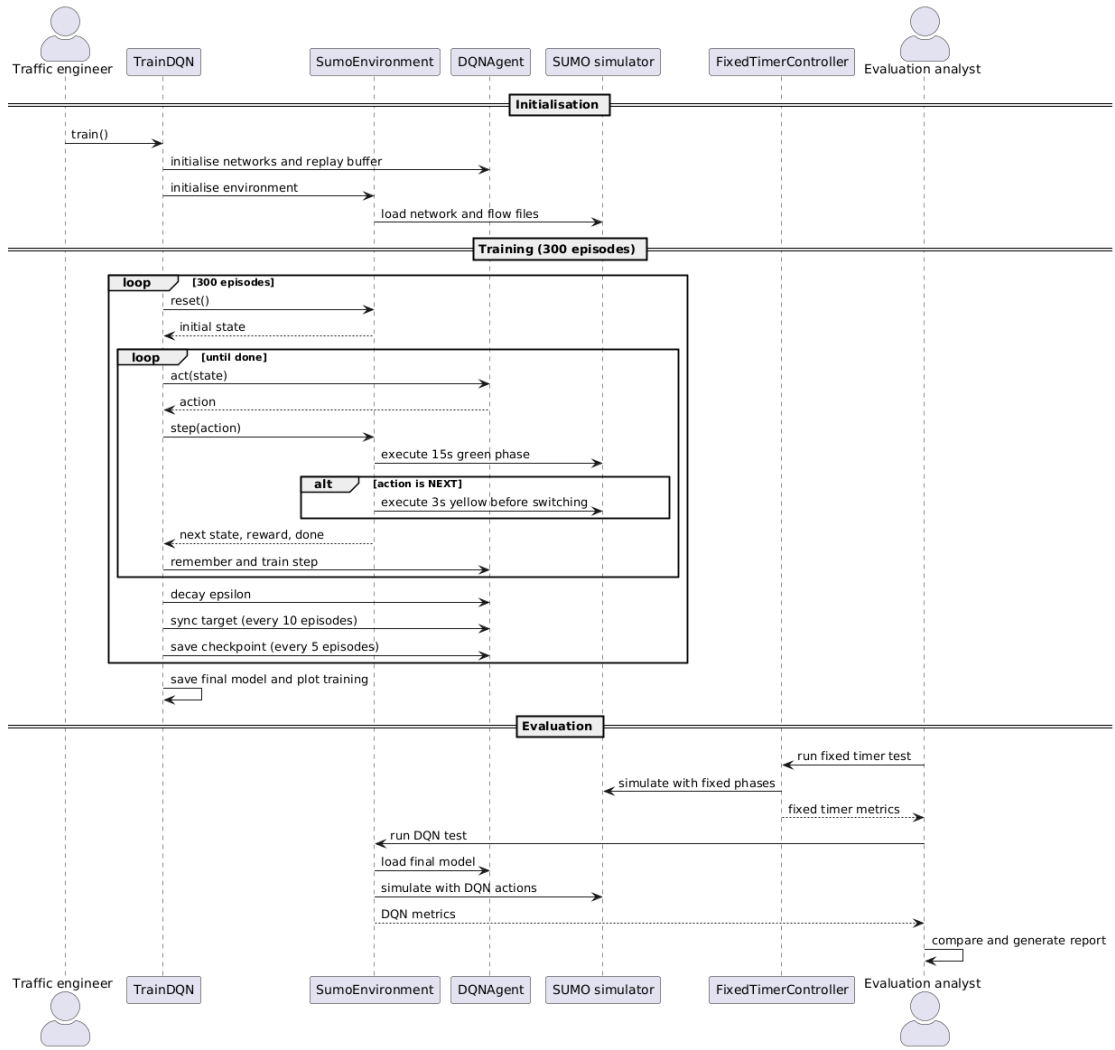


Figure 4-4. Sequence diagram of the Thapathali DQN traffic signal control system

## 4.5. Data Analysis

### 4.5.1. Data Collection

For the traffic network modeling, primary traffic flow data was sourced from the research paper *A Case Study of Thapathali Intersection* by Lav Maharjan [1]. The data collection methodology involved comprehensive field surveys over a duration of five days, strategically selected to capture both weekday (3 days) and weekend (2 days) traffic characteristics. The observations were conducted continuously from 6:00 AM to 10:15 PM, ensuring the inclusion of morning and evening peak periods as well as off-peak transitions.

The data collected during the surveys were represented in terms of PCU Origin-Destination (OD) data and vehicle counts for different time intervals. PCU OD data was recorded for two specific time intervals: one during the morning peak

period and one during the evening peak period, each lasting one hour. These intervals were selected to capture the highest traffic flow during the day. The vehicle counts were recorded separately for these intervals within the given time frame. The representation of the data allows for a clearer analysis of traffic behavior during peak and off-peak times, providing a foundation for modeling the traffic flow more accurately.

#### 4.5.2. Data Representation

Table 4.5.1. Vehicle Class and Adopted PCU Factors

| Vehicle Class | Representation | Adopted PCU Factor |
|---------------|----------------|--------------------|
| Light Vehicle | LV             | 1.5                |
| Heavy Vehicle | HV             | 3                  |
| Bus           | B              | 3                  |
| Mini Bus      | MB             | 2.5                |
| Micro Bus     | MIB            | 1.5                |
| Car           | C              | 1                  |
| Four-Wheel    | F              | 1                  |
| Tempo         | T              | 0.75               |
| Utility       | U              | 0.75               |
| Two-Wheeler   | TW             | 0.25               |

#### 4.5.3. OD Data Allocation

The vehicle allocation system is implemented using a Python script, leveraging libraries like `pandas`, `numpy`, and `matplotlib` for data manipulation and analysis. The following summarizes the key components of the implementation.

The primary input data consists of:

- **Vehicle Count Data (`vehicle_count.xlsx`):** Contains the number of vehicles for various vehicle classes during 15-minute intervals.
- **PCU OD Data (`pcu_od.xlsx`):** Provides the proportions of PCU allocated to each OD pair for AM and PM periods.

#### Data Cleaning

- Non-numeric characters are removed from vehicle count and PCU data.
- Invalid data is replaced with zero.
- Data is converted to numeric format for proper calculation and allocation.
- Vehicles with low counts were grouped into similar vehicle types.
- Vehicles with same PCU and sizes were grouped into one vehicle type.



## Vehicle Allocation Process

For each vehicle class and OD pair, the following process is followed:

- **Proportions Calculation:** AM and PM proportions are calculated based on the PCU data.
- **Allocation via Largest Remainder Method:** Vehicles are initially allocated proportionally, and leftover vehicles are distributed to ODs with the largest fractional values.
- **Piecewise Interpolation:** For each 15-minute interval, vehicle proportions are dynamically interpolated using piecewise interpolation.

## Maternity Restrictions

For certain ODs related to maternity, only specific vehicle classes are allowed (e.g., cars, two-wheelers, four-wheel vehicles, and utility vehicles), based on real-world traffic constraints.

## Output Generation

The final allocated vehicle counts are saved to Excel files, including:

- 15-minute interval counts for every OD and vehicle class.
- A summary sheet showing total vehicles and AM/PM proportions.
- A verification sheet ensuring correct and consistent allocation.

## 4.6. Environment Setup for Traffic Simulation

The first step in the environment setup for traffic simulation using SUMO was to use the NETEDIT tool to create a road network that accurately represents the Thapathali intersection. This tool allows for the design of road networks, including multiple lanes, junctions, and traffic signals. The geometry of the intersection and road structure was designed in NETEDIT and saved as the `map.net.xml` file. This file contains the road network's topography, including nodes (junctions), edges (roads), and their connections. It also defines traffic signals at each intersection, which are crucial for simulating realistic traffic flow.

Once the network structure was defined, vehicle movement was modeled using the `routes.rou.xml` file. This file specifies the routes that vehicles will take through the network and includes information such as departure times, vehicle types, and flow. The vehicle flow is generated to ensure that vehicles enter the network at regular intervals, with a preference for the least congested lane, thus simulating realistic driving behavior.

The `vtypes.xml` file defines various vehicle types used in the simulation, each with specific attributes such as length, maximum speed, acceleration, and deceleration. These characteristics are essential for accurately simulating vehicle behavior across different classes.

The `flows.xml` file captures vehicle flow data and traffic signal phases. This file defines how many vehicles pass through each edge of the network in specific time intervals and the corresponding traffic signal phases.

To interact with the simulation in real-time, the TRACI interface is used. This allows external scripts to control the simulation, modifying vehicle flows and traffic signal phases dynamically. A Python script adjusts the signal phases based on real-time data, such as vehicle counts and congestion levels.

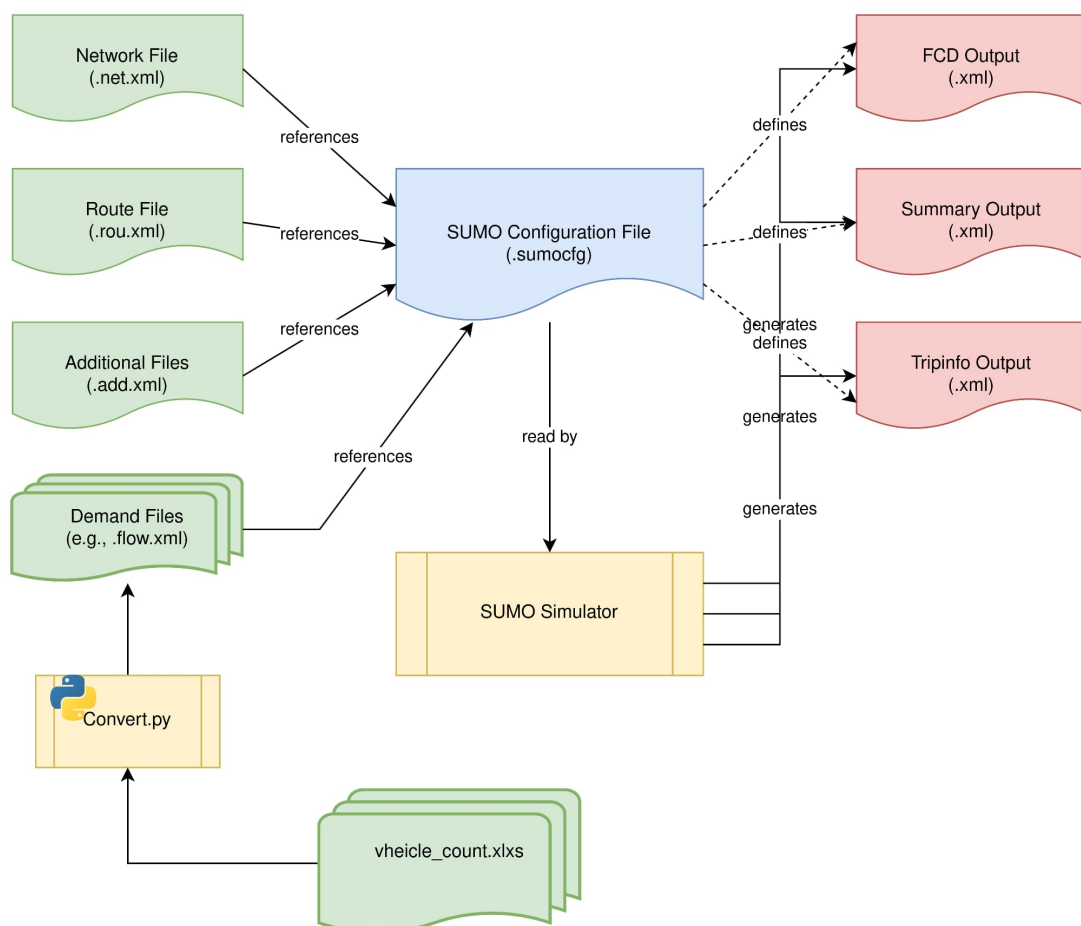


Figure 4-5. SUMO Environment Block Diagram

## 4.7. DQN Agent Development

### 4.7.1. State Representation

The DQN agent observed key traffic conditions at each 15-second decision step, including PCU-weighted vehicle loads on incoming and outgoing lanes, maximum waiting times at stopline lanes, and the current active signal phase. These parameters were combined into a fixed-length 21-dimensional state vector representing the intersection status at each decision step.

$$s_t = \left[ \underbrace{p_1, p_2, \dots, p_7}_{\text{Incoming PCU (7)}}, \underbrace{w_1, w_2, \dots, w_6}_{\text{Stopline Max Wait (6)}}, \underbrace{p_{o1}, p_{o2}, \dots, p_{o5}}_{\text{Outgoing PCU (5)}}, \underbrace{0, 1, 0}_{\text{Current Phase (3)}} \right]$$

### 4.7.2. Action Space

The traffic signal at Thapathali T1 operates across three predefined phases (P1, P2, P3) in a cyclic sequence. At every 15-second decision interval the DQN agent selects one of two actions - Action 1 (STAY) or Action 2 (NEXT) - to control the signal timing. A phase can be held for a maximum of 60 seconds (4 consecutive STAY actions) before automatically advancing to the next phase, ensuring no traffic direction is starved indefinitely.

- **Action 1 (STAY):** The agent maintains the current active phase for another 15 seconds, allowing continued service to the same direction.
- **Action 2 (NEXT):** The agent advances to the next phase in the cycle, switching service to the next direction. Each phase transition includes a 3-second yellow interval to ensure realistic and safe signal timing.

### 4.7.3. Reward Function

A reward function based on Max-Pressure Theory was used. Pressure was calculated as the difference between incoming and outgoing vehicle PCU counts:

$$R = - \left( \sum_{l \in L_{\text{stop}}} \text{PCU}(l) - \sum_{l \in L_{\text{out}}} \text{PCU}(l) \right) - \lambda \sum_{l \in L_{\text{stop}}} \sum_{v \in l} \text{PCU}(v) \cdot w^{acc}(v) \quad (4-1)$$

**Where:**

- $L_{\text{stop}}$  - set of 6 stopline lanes
- $L_{\text{out}}$  - set of 5 outgoing lanes
- $\text{PCU}(l)$  - total PCU-weighted vehicle load on lane  $l$  at current step

- $PCU(v)$  - PCU value of individual vehicle  $v$
- $w^{acc}(v)$  - accumulated waiting time (seconds) of vehicle  $v$  since simulation start
- $\lambda = 0.0004$  - weighting factor balancing throughput and wait time penalty

#### 4.8. Design Justification

Several reward formulations were considered before adopting the Max-Pressure based reward. A pure delay-based reward penalising total accumulated waiting time across all vehicles was ruled out because accumulated waiting time grows unboundedly within an episode, causing Q-value targets to grow without bound and leading to loss divergence during training. A throughput-only reward rewarding the agent purely for vehicles cleared was also considered but rejected because outgoing lane flow at Thapathali is heavily influenced by road width and geometry rather than signal decisions alone, meaning the agent would learn to favour wider outgoing lanes regardless of which approach actually had the heaviest queue.

The adopted formulation is grounded in Max-Pressure theory, which has theoretical throughput optimality properties under certain demand conditions and has been validated in prior DRL traffic signal control literature. The first term computes the pressure difference between stopline PCU accumulation and outgoing PCU discharge - directly measuring whether the agent's current phase choice is clearing vehicles efficiently. The second term adds a PCU-weighted accumulated waiting time penalty scaled by  $\lambda = 0.0004$ , discouraging the agent from ignoring any single approach for extended periods. The  $\lambda$  value was calibrated to keep the wait penalty secondary to the throughput objective, preventing the agent from sacrificing overall flow in favour of minimising wait time on a single approach.

##### 4.8.1. Impact on Learning

The reward function directly shapes the Q-value targets the agent learns from at every training step. The pressure term  $-(\sum_{l \in L_{stop}} PCU(l) - \sum_{l \in L_{out}} PCU(l))$  provides an instantaneous signal that reflects the current state of the intersection - when the agent selects the correct phase, outgoing PCU increases and stopline PCU decreases, producing a less negative reward that reinforces the action. When the agent holds the wrong phase, stopline PCU accumulates and outgoing PCU stays low, producing a more negative reward that discourages the action. This direct cause-and-effect relationship between phase selection and reward is essential for stable Q-value learning.

The accumulated wait penalty  $\lambda \sum_v PCU(v) \cdot w^{acc}(v)$  complements the pressure term by introducing a fairness objective-the agent is penalised for vehicles that have been waiting for extended periods regardless of which lane they occupy. This prevents the

agent from learning a degenerate policy that maximises throughput on one high-flow approach while consistently starving lower-volume approaches of green time.

Together the two terms ensure that the agent learns to balance both efficiency -clearing the maximum number of PCU per unit time - and equity - preventing excessive delay accumulation on any single approach. This dual objective aligns directly with the project's performance metrics of average wait time reduction and throughput efficiency improvement, ensuring that what the agent optimises during training corresponds to what is measured during evaluation.

#### 4.8.2. Neural Network Architecture

The agent was implemented using a deep neural network with one input layer, three hidden layers of 128, 64, and 32 neurons respectively, and an output layer with two action values corresponding to STAY and NEXT. The input layer accepts the 21-dimensional state vector representing current intersection conditions. ReLU activation functions were applied after each hidden layer to introduce non-linearity. The output layer produces Q-values for each of the two possible actions, from which the agent selects the action with the highest estimated future reward.

Two identical networks were used: a Q-network for action selection and a target network to stabilise learning during training. The target network weights were updated every 10 episodes by copying the Q-network weights, preventing oscillations and divergence during the training process.

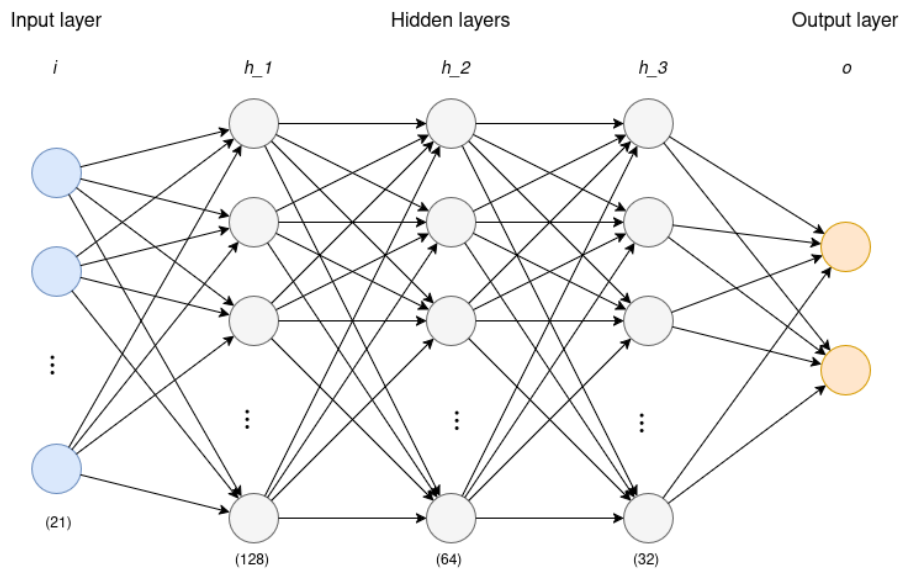


Figure 4-6. Neural Network Architecture for Q-value Prediction

## **4.9. Hyperparameter Selection and Sensitivity**

The DQN agent’s behaviour is governed by a set of hyperparameters whose values were selected based on a combination of established practice in the DRL traffic signal control literature and the specific operational constraints of the Thapathali intersection. Each variable is justified below in terms of its functional role and the rationale for its assigned value.

### **4.9.1. State Dimensionality (21 dimensions)**

The state vector encodes three categories of intersection information: PCU-weighted load on 7 origin lanes, maximum waiting time on 6 stopline lanes, PCU-weighted load on 5 outgoing lanes, and a 3-dimensional one-hot encoding of the active phase. Each component addresses a distinct informational need. The origin lane PCU values tell the agent where pressure is building; the stopline waiting times capture fairness - whether any single approach has been ignored for too long; the outgoing PCU values reflect whether the intersection is successfully discharging traffic downstream; and the phase encoding prevents the agent from selecting actions without knowing its current operating context. Omitting any of these components would structurally impair the agent’s ability to make informed decisions. For instance, an agent without outgoing lane information cannot distinguish between a green phase that is successfully clearing traffic and one that is being blocked by downstream congestion.

### **4.9.2. Replay Buffer Capacity (10,000 transitions)**

The replay buffer stores past experience tuples to break temporal correlations between consecutive training samples. A buffer that is too small would cause the agent to train predominantly on recent experiences, leading to overfitting to current traffic conditions and instability when demand patterns shift between hours. A buffer that is too large relative to the number of training steps would dilute recent experience with outdated transitions, slowing adaptation. At 10,000 transitions and a 15-second decision step, this buffer spans approximately 41 simulated hours of experience - sufficient to cover the full daily demand cycle multiple times across episodes while remaining computationally tractable. Increasing this to 50,000 would likely improve sample diversity but would require proportionally more training episodes to populate and exploit the buffer effectively.

### **4.9.3. Batch Size (64 transitions)**

Mini-batch gradient updates use 64 randomly sampled transitions per training step, a value standard in DQN implementations. Smaller batches introduce higher gradient variance, potentially destabilising Q-value updates. Larger batches reduce variance

but increase per-step computational cost and can cause the network to overfit to the current batch distribution before the buffer has been adequately repopulated. At 64, each update covers a representative cross-section of stored experiences without excessive computational overhead on CPU-based training.

#### **4.9.4. Discount Factor $\gamma = 0.95$**

The discount factor controls how much the agent values future rewards relative to immediate ones. A value of 1.0 would make the agent treat all future rewards as equally important regardless of how far ahead they occur, which is unsuitable for traffic control where the causal link between a signal decision and its traffic outcome attenuates over time as other vehicles arrive and depart. A value near 0 would make the agent myopic, optimising only the immediate reward of each 15-second step without accounting for downstream queue effects. At  $\gamma = 0.95$ , a reward received five decision steps (75 seconds) ahead is discounted to approximately 77% of its face value - a horizon long enough to capture the queue-clearing consequences of phase decisions while keeping the training objective stable. Reducing  $\gamma$  to 0.90 would likely cause the agent to be more reactive and less capable of holding a phase strategically to clear a building queue.

#### **4.9.5. Epsilon Start ( $\varepsilon = 1.0$ ), Minimum ( $\varepsilon_{\min} = 0.01$ ), and Decay ( $\varepsilon_{\text{decay}} = 0.985$ )**

The epsilon-greedy strategy governs exploration versus exploitation. Starting at  $\varepsilon = 1.0$  ensures the agent explores the state space broadly in early episodes rather than immediately exploiting the random initial Q-values of the untrained network, which would bias learning toward the first arbitrary policy encountered. The decay factor of 0.985 applied after each episode causes  $\varepsilon$  to reach approximately 0.18 by episode 100 and 0.01 by episode 300, providing a gradual transition from exploration to exploitation that aligns with the agent's growing ability to make informed decisions as training progresses. A faster decay (e.g., 0.97) would cause the agent to commit to exploitation prematurely, before Q-values have stabilised, risking convergence to a suboptimal policy. A slower decay (e.g., 0.995) would extend random exploration unnecessarily into late training, wasting episodes where the agent has already learned a reasonable policy. The minimum  $\varepsilon = 0.01$  retains a 1% probability of random action throughout evaluation to prevent the agent from becoming trapped in a deterministic but suboptimal cycle.

#### **4.9.6. Target Network Update Frequency (every 10 episodes)**

The target network provides stable Q-value targets during training. If the target network updated at every step alongside the Q-network, the training targets would shift continuously, creating a moving-target problem that frequently causes Q-value divergence. Updating too infrequently (e.g., every 50 episodes) would cause the agent to

train against outdated targets for too long, particularly during the rapid early learning phase. At 10 episodes, the target network is refreshed frequently enough to reflect meaningful policy improvement while remaining stable enough to provide reliable training signals between updates.

#### **4.9.7. Reward Weighting Factor $\lambda = 0.0004$**

The reward function combines a Max-Pressure throughput term with a PCU-weighted accumulated waiting time penalty scaled by  $\lambda$ . The role of  $\lambda$  is to determine how much the agent penalises individual vehicle waiting time relative to the aggregate pressure objective. Without the waiting time term ( $\lambda = 0$ ), the agent could learn a policy that maximises overall throughput while consistently starving one low-volume approach of green time - technically efficient but operationally unacceptable. Too large a  $\lambda$  (e.g., 0.01) would cause the agent to focus disproportionately on eliminating individual vehicle delay, potentially sacrificing overall throughput. The value  $\lambda = 0.0004$  was selected to keep the wait penalty secondary in magnitude to the pressure term across typical PCU load ranges observed at Thapathali, so that fairness acts as a corrective constraint rather than the primary objective. At typical peak-hour stopline loads of 20–30 PCU and accumulated wait times of 30–60 seconds per vehicle, the wait penalty contributes approximately 0.24–0.72 units per vehicle to the reward, while the pressure term contributes 5–15 units, maintaining the intended hierarchy. Increasing  $\lambda$  by an order of magnitude would be expected to reduce the worst-case wait times on low-volume approaches but at the cost of overall throughput efficiency.

#### **4.9.8. Decision Step Duration (15 seconds)**

Each action is held for 15 seconds of simulation time before the agent re-evaluates the intersection state. This value represents a balance between responsiveness and stability. A shorter step (e.g., 5 seconds) would allow more frequent decisions but would increase the risk of unnecessary phase switching, as queue dynamics at Thapathali do not change meaningfully within a few seconds. Frequent switching also incurs additional yellow-phase transition time (3 seconds each), which would consume a disproportionate share of available green time. A longer step (e.g., 30 seconds) would reduce switching overhead but would limit the agent's ability to respond to rapid changes in approach demand. At 15 seconds, the agent can make approximately 4 decisions per minute - sufficient granularity to track peak-hour demand dynamics without excessive transition overhead. The maximum phase duration of 60 seconds (4 consecutive STAY actions) similarly prevents any single approach from being served indefinitely while still allowing the agent to hold a productive phase through a full queue-clearing cycle.



#### **4.9.9. Yellow Phase Duration (3 seconds)**

A 3-second yellow interval is inserted before every phase transition to reflect safe and realistic signal operation. This value is consistent with standard traffic engineering practice for intersection approach speeds in the 25–40 km/h range typical of urban Kathmandu roads. Reducing this to 1–2 seconds would be unsafe; increasing it to 5 seconds would unnecessarily consume green time, particularly at the 15-second decision step resolution where a 5-second yellow would represent one-third of available action time.

#### **4.9.10. Hidden Layer Architecture (128 → 64 → 32 neurons)**

The three-layer architecture with progressively narrowing hidden layers is designed to extract hierarchical representations from the 21-dimensional state input. The first layer (128 neurons) provides sufficient capacity to capture the full range of pairwise interactions between state features - lane loads, waiting times, and phase indicators - that jointly determine the optimal action. The subsequent layers (64 and 32 neurons) compress this representation into increasingly abstract features, forcing the network to generalise rather than memorise specific state patterns. A deeper or wider network could in principle capture more complex relationships but would require substantially more training data and episodes to converge reliably, which was not feasible within the 300-episode training budget. A shallower network (e.g., single hidden layer) risks underfitting the non-linear structure of traffic state dynamics.

#### **4.9.11. Learning Rate ( $\alpha = 0.001$ ) for Adam Optimiser**

The Adam optimiser's learning rate controls the step size of each gradient update to the Q-network weights. At 0.001, a standard default for Adam in DRL applications, the network updates are large enough to make meaningful progress per batch but small enough to avoid overshooting well-converged Q-value estimates. A higher learning rate (e.g., 0.01) risks Q-value oscillations and divergence, particularly during the early high-variance exploration phase. A lower rate (e.g., 0.0001) would require significantly more training episodes to converge.

#### **4.9.12. Number of Training Episodes (300)**

The training run of 300 episodes was selected based on the epsilon decay schedule: at  $\epsilon_{\text{decay}} = 0.985$  per episode,  $\epsilon$  reaches its minimum of 0.01 by approximately episode 300, meaning the agent transitions fully to exploitation at the same point training concludes. Extending training beyond 300 episodes would primarily refine the policy under full exploitation, offering diminishing returns unless accompanied by renewed exploration. The episode reward trajectory in Figure 6-2 confirms that the moving average stabilises by

approximately episode 250, supporting the adequacy of 300 episodes for this intersection configuration.

## **4.10. Training Process**

### **4.10.1. Learning Strategy**

An  $\epsilon$ -greedy learning strategy was used to balance exploration and exploitation during training. The agent initially explored the environment by selecting random actions with  $\epsilon$  set to 1.0, ensuring broad coverage of the state space in early episodes. After each episode,  $\epsilon$  was multiplicatively decayed by a factor of 0.985, gradually reducing random exploration and allowing the agent to increasingly rely on its learned Q-values for action selection. By approximately episode 300,  $\epsilon$  reached its minimum value of 0.01, at which point the agent exploits its learned policy for 99% of decisions while retaining a small probability of random exploration to avoid getting trapped in suboptimal policies.

### **4.10.2. Experience Replay**

Experience replay improved stability by storing transitions in a 10,000-capacity buffer. Random samples were used for training to break temporal correlations.

### **4.10.3. Training Loop**

The model was trained over 300 episodes. The process included:

1. Calculate Target Q:  $Q_{\text{target}} = r + \gamma \times \max(Q(s', a'))$ .
2. Calculate Current Q:  $Q_{\text{current}} = Q(s, a)$ .
3. Backpropagate Loss:  $\text{MSE}(Q_{\text{target}}, Q_{\text{current}})$ .
4. Update policy network weights.

## **4.11. Algorithms Used**

---

**Algorithm 4.11.1** Reward Calculation

---

**Require:** Stopline lane PCU values, Outgoing lane PCU values, Vehicle waiting times

**Ensure:** Reward value

```
1: procedure CALCULATEREWARD
2:   incoming_pcu  $\leftarrow \sum_{l \in L_{\text{stop}}} \text{PCU}(l)$ 
3:   outgoing_pcu  $\leftarrow \sum_{l \in L_{\text{out}}} \text{PCU}(l)$ 
4:   weighted_wait  $\leftarrow 0$ 
5:   for all  $l \in L_{\text{stop}}$  do
6:     for all  $v \in l$  do
7:       weighted_wait  $\leftarrow \text{weighted\_wait} + \text{PCU}(v) \cdot w^{\text{acc}}(v)$ 
8:     end for
9:   end for
10:  reward  $\leftarrow -(\text{incoming\_pcu} - \text{outgoing\_pcu}) - \lambda \cdot \text{weighted\_wait}$ 
11:  return reward
12: end procedure
```

---

---

**Algorithm 4.11.2** DQN Training Process

---

**Require:** Number of episodes  $E = 300$ , Replay buffer capacity  $B = 10000$ , Target network update frequency  $F = 10$ , Batch size = 64

**Ensure:** Trained Q-network

```
1: procedure TRAINDQN( $E, B, F$ )
2:   Initialize  $\epsilon \leftarrow 1.0$ ,  $\epsilon_{min} \leftarrow 0.01$ ,  $\epsilon_{decay} \leftarrow 0.985$ 
3:   Initialize replay buffer with capacity  $B$ 
4:   Initialize Q-network and target network with random weights
5:   for  $episode \leftarrow 1$  to  $E$  do
6:     Reset environment
7:      $state \leftarrow \text{GetInitialState}()$ 
8:     while simulation is running do
9:        $action \leftarrow \text{SelectAction}(state, \epsilon)$  ▷  $\epsilon$ -greedy selection
10:      Execute  $action$  for 15 seconds
11:       $reward \leftarrow \text{CalculateReward}()$ 
12:       $next\_state \leftarrow \text{GetNextState}()$ 
13:      Store  $(state, action, reward, next\_state)$  in replay buffer
14:      if replay buffer has  $\geq 64$  samples then
15:        Sample random batch of 64 from replay buffer
16:        Calculate target:  $Q_{target} = r + \gamma \times \max_{a'}(Q(s', a'))$ 
17:        Calculate current:  $Q_{current} = Q(s, a)$ 
18:        Compute loss:  $\mathcal{L} = \text{MSE}(Q_{target}, Q_{current})$ 
19:        Update Q-network weights via backpropagation
20:      end if
21:       $state \leftarrow next\_state$ 
22:    end while
23:     $\epsilon \leftarrow \max(\epsilon_{min}, \epsilon \times \epsilon_{decay})$  ▷ Decay exploration rate
24:    if  $episode \bmod F = 0$  then
25:      Update target network weights from Q-network
26:    end if
27:    if  $episode \bmod 5 = 0$  then
28:      Save model checkpoint
29:    end if
30:  end for
31:  return trained Q-network
32: end procedure
```

---

## 5. IMPLEMENTATION DETAILS

This project focuses on developing an intelligent traffic control system using DQN to optimize traffic flow at the Thapathali Intersection in Kathmandu. By applying DQN, the system dynamically adjusts traffic light phases based on real-time data such as vehicle density, waiting times, and congestion levels. The primary objective is to minimize vehicle wait times and enhance traffic throughput by learning and implementing the most efficient signal phases.

The system utilizes SUMO for simulating traffic scenarios and TRACI to enable real-time interaction between the DQN model and the traffic simulation. This integration allows the DQN model to control traffic signal phases, assess the impact of its actions, and continuously improve the traffic flow based on the simulation feedback.

### 5.1. Vehicle Flow Generation

---

**Algorithm 5.1.1** Perfect Vehicle Allocation with Remainder Tracking

---

```
1: Input:
2:   vehicle_counts: Dictionary  $\{vc_1 : count_1, vc_2 : count_2, \dots\}$ 
3:   proportions: Dictionary  $\{od_1 : p_1, od_2 : p_2, \dots\}$ 
4:   od_names: List of OD names  $[od_1, od_2, \dots, od_N]$ 
5:   prev_remainders: Dictionary  $\{od_1 : r_1, od_2 : r_2, \dots\}$  (Optional)
6: Output:
7:   allocation: Dictionary  $\{od_1 : \{vc_1 : count, \dots\}, \dots\}$ 
8:   new_remainders: Dictionary  $\{od_1 : r_1, od_2 : r_2, \dots\}$ 
9: procedure ALLOCATEINTERVALPERFECT(vehicle_counts, proportions,
   od_names, prev_remainders)
10:   if prev_remainders is None then
11:     Initialize prev_remainders  $\leftarrow \{od : 0 \text{ for all } od \in od\_names\}$ 
12:   end if
13:   Initialize allocation  $\leftarrow \{od : \{\}$  for all  $od \in od\_names\}$ 
14:   Initialize new_remainders  $\leftarrow \{od : 0 \text{ for all } od \in od\_names\}$ 
15:   for all (vehicle_class, total_count)  $\in vehicle\_counts$  do
16:     if total_count = 0 then
17:       continue
18:     end if
19:     for all od  $\in od\_names$  do
20:       exact  $\leftarrow total\_count \times proportions[od] + prev\_remainders[od]$ 
21:       allocated  $\leftarrow \lfloor exact \rfloor$ 
22:       remainder  $\leftarrow exact - allocated$ 
23:       allocation[od][vehicle_class]  $\leftarrow allocated$ 
24:       new_remainders[od]  $\leftarrow remainder$ 
25:     end for
26:   end for
27:   return (allocation, new_remainders)
28: end procedure
```

---

---

**Algorithm 5.1.2** Batch Perfect Allocation using piecewise linear interpolation

---

```
1: procedure ALLOCATEVEHICLES PERFECT(vehicle_counts, proportions,  
   od_names)  
2:   Initialize result  $\leftarrow \{od : \{vc : 0 \text{ for all } vc\} \text{ for all } od \in od\_names\}$   
3:   for all (vehicle_class, total_count)  $\in vehicle\_counts$  do  
4:     if total_count = 0 then  
5:       continue  
6:     end if  
7:     targets  $\leftarrow \{\}$  ▷ Exact target allocations  
8:     for all od  $\in od\_names$  do  
9:       targets[od]  $\leftarrow total\_count \times proportions[od]$   
10:    end for  
11:    ▷ Step 1: Allocate integer parts  
12:    allocated  $\leftarrow \{\}$   
13:    fractions  $\leftarrow \{\}$   
14:    for all od  $\in od\_names$  do  
15:      allocated[od]  $\leftarrow \lfloor targets[od] \rfloor$   
16:      fractions[od]  $\leftarrow targets[od] - allocated[od]$   
17:    end for  
18:    total_allocated  $\leftarrow \sum_{od \in od\_names} allocated[od]$   
19:    remaining  $\leftarrow total\_count - total\_allocated$   
20:    if remaining > 0 then  
21:      ▷ Step 2: Sort ODs by fractional part (descending)  
22:      sorted_ods  $\leftarrow \text{SORT}(fractions.items(), \text{by value descending})$   
23:      for i  $\leftarrow 0$  to min(remaining, |od_names|) - 1 do  
24:        od  $\leftarrow sorted\_ods[i].key$   
25:        allocated[od]  $\leftarrow allocated[od] + 1$   
26:      end for  
27:    end if  
28:    ▷ Store final allocation for this vehicle class  
29:    for all od  $\in od\_names$  do  
30:      result[od][vehicle_class]  $\leftarrow allocated[od]$   
31:    end for  
32:  end for  
33:  return result  
34: end procedure
```

---

---

**Algorithm 5.1.3** Mathematical Formulation of largest remainder method for vehicle allocation

---

**AllocateIntervalPerfect:**

Given:  $v_c$  (vehicles of class  $c$ ),  $p_n$  (proportion for OD  $n$ ),  $r_n^{prev}$  (previous remainder)  
 Compute:  $exact_n = v_c \cdot p_n + r_n^{prev}$   
 $a_n = \lfloor exact_n \rfloor$   
 $r_n^{new} = exact_n - a_n$   
 Result:  $\sum_n a_n = v_c$  (when summed over all classes with remainder tracking)

**AllocateVehiclesPerfect (Largest Remainder Method):**

Given:  $v_c$  (total vehicles of class  $c$ ),  $p_n$  (proportion for OD  $n$ )  
 Step 1:  $a_n^{(1)} = \lfloor v_c \cdot p_n \rfloor$   
 $f_n = v_c \cdot p_n - a_n^{(1)}$   
 Step 2: Sort  $\{f_n\}$  in descending order  
 Step 3:  $a_n = a_n^{(1)} + \begin{cases} 1 & \text{if } n \text{ is among top } R \text{ ODs} \\ 0 & \text{otherwise} \end{cases}$   
 Where:  $R = v_c - \sum_n a_n^{(1)}$

**Conservation Property:**

$$\sum_{n=1}^N a_n = v_c \quad \forall c$$


---

## 5.2. Traffic Simulation Environment (SUMO)

To simulate the traffic environment and test our DQN-based traffic signal control system, we used SUMO, an open-source tool for traffic modeling. Below are the steps and configurations involved in setting up the simulation.

### 5.2.1. SUMO Configuration Files

The implementation of SUMO environment involved the fine-tuning of various parameters to optimize the traffic signal control for the Thapathali intersection. The following parameters were modified or explicitly set in the SUMO configuration files. All parameters that were not explicitly specified or tuned in the configuration files default to SUMO's standard settings. This ensures that the simulation remains consistent with SUMO's standard behavior while allowing for the fine-tuning and optimization necessary for this project [6].

#### 5.2.2. `map.net.xml`

This file is generated by NETEDIT and contains the road network's topology. It defines the layout of roads (edges) and intersections (nodes), including information such as lanes, traffic signals, and the geometry of the road network and left-side driving was enabled.

#### 5.2.3. `routes.rou.xml`

This file defines the vehicle routes and traffic flow in the simulation. It specifies how many vehicles will be on the road and the routes they take through the network.

#### 5.2.4. `vtypes.xml`

This file specifies the characteristics of different vehicle types used in the simulation. Each `<vType>` tag describes a vehicle's properties, such as length, width, acceleration, deceleration, maximum speed, and other factors relevant to vehicle behavior in the simulation.

- **vClass:** Specifies the type of vehicle (e.g., "delivery", "truck", "bus", "passenger", "evehicle", "motorcycle").
- **length:** The length of the vehicle in meters.
- **width:** The width of the vehicle in meters.
- **accel:** The vehicle's acceleration in  $\text{m/s}^2$ .
- **decel:** The vehicle's deceleration rate in  $\text{m/s}^2$ .
- **emergencyDecel:** The emergency deceleration rate in  $\text{m/s}^2$ .
- **maxSpeed:** The vehicle's maximum speed in  $\text{m/s}$ .



- **minGap:** The minimum gap between vehicles in meters.
- **guiShape:** Specifies the visual representation of the vehicle.
- **sigma:** Influences the vehicle's randomness in lane-changing behavior.
- **tau:** Represents the reaction time of the vehicle.

Table 5.2.1. Vehicle Type Configurations in SUMO Simulation

| Attribute                           | LV        | HV        | B                | MB       | MIB        |
|-------------------------------------|-----------|-----------|------------------|----------|------------|
| Class                               | Delivery  | Truck     | Bus              | Bus      | Delivery   |
| Length (m)                          | 6.2       | 8.0       | 10.85            | 6.6      | 5.4        |
| Width (m)                           | 2.16      | 2.4       | 2.5              | 2.2      | 2.0        |
| Acceleration (m/s <sup>2</sup> )    | 2.6       | 1.3       | 1.2              | 2.6      | 2.6        |
| Deceleration (m/s <sup>2</sup> )    | 4.5       | 4.0       | 4.0              | 4.5      | 4.5        |
| Emergency Decel (m/s <sup>2</sup> ) | 7.0       | 7.0       | 7.0              | 9.0      | 9.0        |
| Max Speed (m/s)                     | 6.14      | 4.83      | 4.83             | 5.66     | 5.66       |
| Min Gap (m)                         | 1         | 1.5       | 1.5              | 1        | 1          |
| Sigma                               | 0.4       | 0.2       | 0.3              | 0.4      | 0.4        |
| Tau                                 | 1.5       | 3.0       | 2.5              | 2.0      | 1.5        |
| GUI Shape                           | Delivery  | Truck     | Bus              | Bus      | Delivery   |
| Attribute                           | C         | F         | U                | T        | TW         |
| Class                               | Passenger | Passenger | Electric Vehicle | Delivery | Motorcycle |
| Length (m)                          | 4.0       | 4.6       | 4.6              | 3.6      | 1.8        |
| Width (m)                           | 1.8       | 1.8       | 1.8              | 1.6      | 0.9        |
| Acceleration (m/s <sup>2</sup> )    | 2.6       | 2.6       | 2.6              | 2.6      | 6.0        |
| Deceleration (m/s <sup>2</sup> )    | 4.5       | 4.5       | 4.5              | 4.5      | 10.0       |
| Emergency Decel (m/s <sup>2</sup> ) | 9.0       | 9.0       | 9.0              | 9.0      | 10.0       |
| Max Speed (m/s)                     | 6.86      | 6.14      | 6.86             | 7.88     | 9.25       |
| Min Gap (m)                         | 1         | 1         | 1                | 1        | 1          |
| Sigma                               | 0.5       | 0.5       | 0.5              | 0.3      | 0.6        |
| Tau                                 | 1.2       | 1.2       | 1.4              | 1.2      | 1.0        |
| GUI Shape                           | Passenger | Passenger | Electric Vehicle | Delivery | Motorcycle |

### 5.2.5. flows.xml

In our simulation, the `departLane="best"` parameter was employed to ensure that vehicles are assigned to the most optimal lane when they enter the network. This helps to improve the overall efficiency of the traffic flow by minimizing congestion and ensuring that vehicles are distributed across available lanes in an optimal manner.

Additionally, a uniform vehicle generation pattern was applied, with vehicles entering the network in 15-minute intervals. This ensures that traffic flow is consistent and realistic, mirroring typical traffic patterns observed during peak hours while maintaining a balanced vehicle entry rate across the simulation period.

### 5.2.6. `simulation.sumocfg` (SUMO Configuration File)

The SUMO configuration file defines the simulation parameters, such as which files to use (network, route, vtype, flows, and add), the simulation duration, and various settings for the simulation itself. The lateral-resolution parameter in the SUMO configuration file (`.sumocfg`) is used to define the resolution at which the lanes are subdivided during the simulation. It specifies the width of the lane in terms of the vehicle's lateral position. This value is particularly important for controlling the precision of lane-changing behavior, vehicle positioning, and simulation accuracy in SUMO.

To ensure high accuracy in vehicle positioning and lane-changing behavior, a lateral resolution of 0.2 meters was used in the SUMO simulation.

### 5.3. Environment Wrapper Implementation

The `SumoEnvironment` class provided the interface between the SUMO simulation and the reinforcement learning agent. The `reset()` method initialised each episode by restarting the SUMO simulation and resetting all internal state variables. The `step(action)` method executed the selected traffic signal action over a 15-second decision window, collecting rewards at each simulation step and returning the averaged reward, next state, and episode metrics upon completion. The `_get_state()` method extracted the 21-dimensional state vector comprising PCU-weighted vehicle loads on origin and outgoing lanes, maximum waiting times on stopline lanes, and a one-hot encoding of the current active phase. The `_calculate_reward()` method computed the reward by comparing stopline PCU against outgoing PCU discharge and adding a PCU-weighted accumulated waiting time penalty. The traffic light phases were controlled through SUMO's TraCI interface, with phase transitions managed internally by the environment.

The state representation used 21 dimensions: 7 origin lane PCU values reflecting approaching traffic volume, 6 stopline lane maximum waiting times capturing queue severity at the signal, 5 outgoing lane PCU values reflecting current discharge flow through the intersection, and 3 phase indicators encoded as one-hot vectors representing the currently active phase (P1, P2, P3). The action space consisted of 2 discrete actions - STAY and NEXT - operating over a cyclic sequence of three signal phases.

The reward function followed Max-Pressure theory. For each incoming lane, pressure was calculated as incoming vehicle count minus average outgoing vehicle count. The total reward was the negative sum of all lane pressures, encouraging congestion reduction.

## 5.4. Traffic Flow and Signal Phase Design at Thapathali Intersection

### 5.4.1. Traffic Flow

The traffic movements at the **Thapathali Intersection** are organized based on the approach roads and controlled through designated signal phases. These movements include straight, left-turn, right-turn, and bypass flows, each corresponding to specific paths within the intersection. The movement directions are defined relative to the forward direction of travel associated with each signal phase.

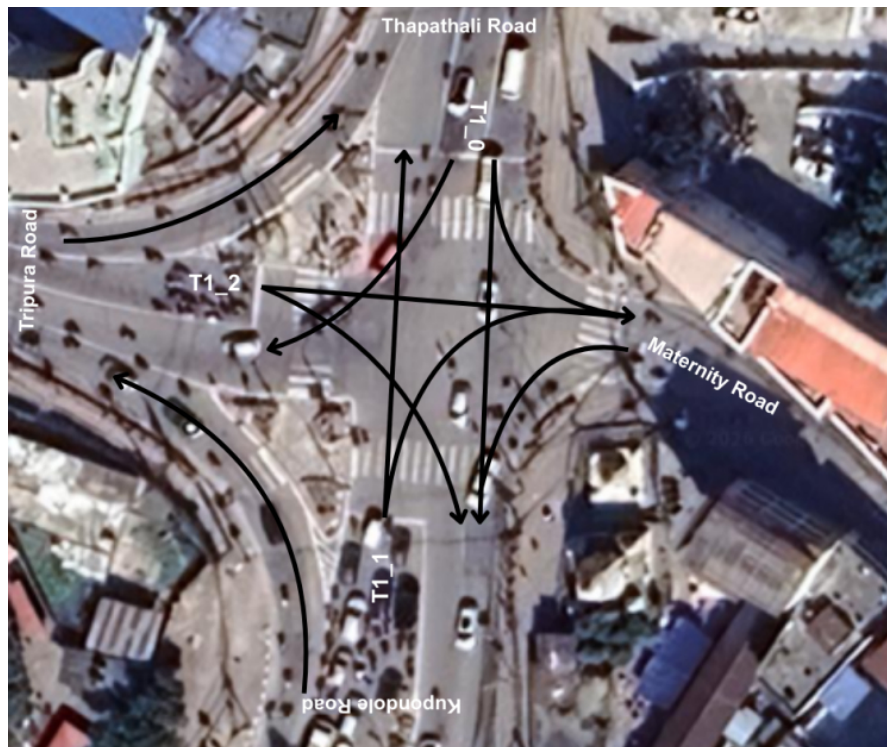


Figure 5-1. Thapathali Road Traffic flow and Traffic light control stage representation

#### Traffic Flow from Tripura Marg

Vehicles approaching from Tripura Marg have the following movement options:

- Bypass to Thapathali Road (free-flow movement without signal control)
- Straight movement to Maternity Road via signal phase T1\_2
- Right turn to Kupondole Road via signal phase T1\_2

#### Traffic Flow from Kupondole Road

Vehicles entering from Kupondole Road can proceed as follows:

- Bypass to Tripura Marg (uncontrolled/free movement)
- Right turn to Maternity Road via signal phase T1\_1

- Straight movement (continuation) via signal phase T1\_1

#### **Traffic Flow from Maternity Road**

Vehicles approaching from Maternity Road have:

- Left turn to Kupondole Road

#### **Traffic Flow from Thapathali Road**

Vehicles from Thapathali Road are allowed the following movements:

- Right turn to Tripura Marg via signal phase T1\_0
- Left turn to Maternity Road via signal phase T1\_0
- Straight movement to Kupondole Road via signal phase T1\_0

#### **Signal Phase Representation**

The signal phase identifiers T1\_0, T1\_1, and T1\_2 represent distinct traffic light control stages within the intersection. Each phase governs a specific set of movements from a particular approach, ensuring orderly and conflict-free traffic flow. These phase labels are commonly used in traffic simulation environments such as SUMO to define internal connection paths and coordinate signal timing with vehicle movements.

#### **5.4.2. Signal Phase Design at Thapathali Intersection**

The traffic signal system at the Thapathali Intersection is organized into three primary movement groups: T1\_0, T1\_1, and T1\_2. These represent distinct sets of directional flows (left, through, right) that are activated in different signal phases. The system operates under three main phase configurations: P0, P1, and P2, where each phase determines which movements are allowed (green) and which are restricted (red).

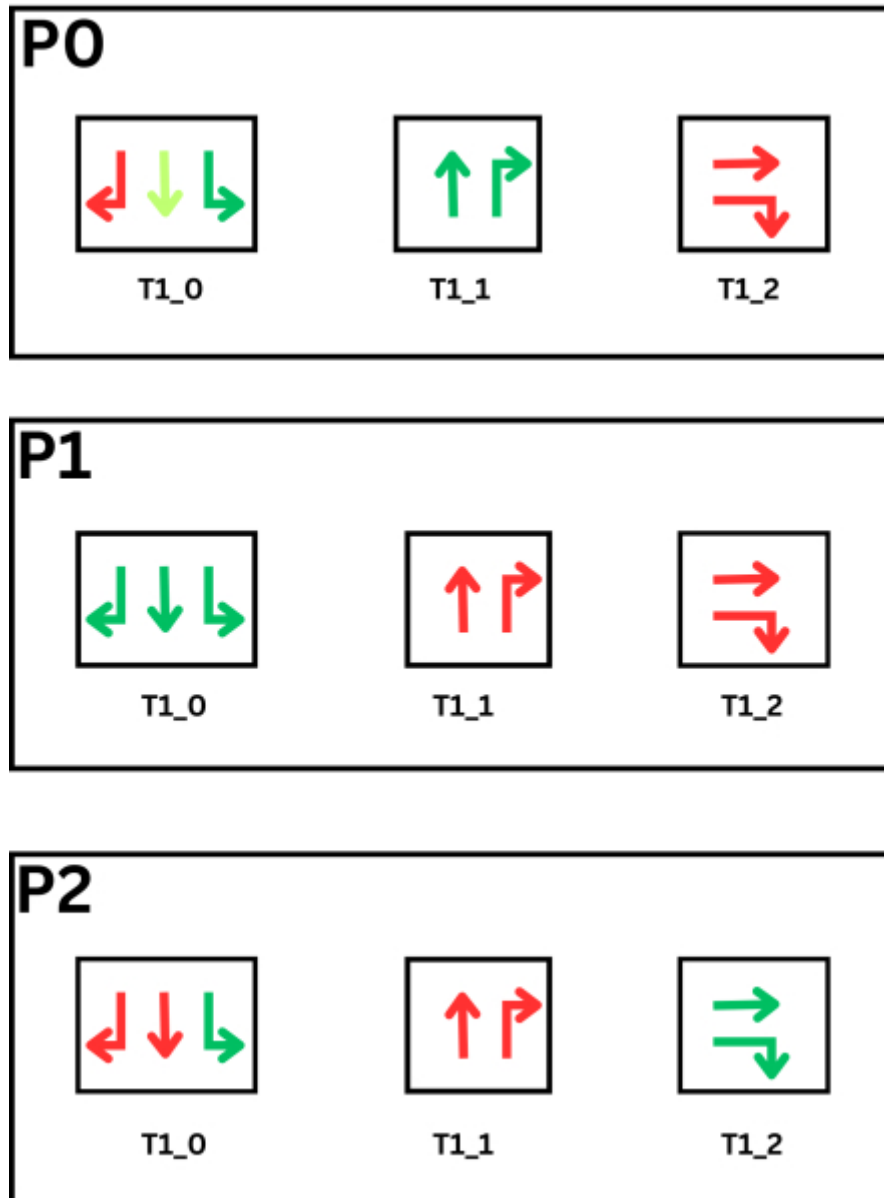


Figure 5-2. Implemented Traffic Signal Phases for fixed-timer and DQN-based Agent

## Traffic Signal Phases

### Phase P0

In Phase P0, traffic flow is primarily allowed from T1\_1, while movements from T1\_2 are fully restricted and T1\_0 is partially permitted.

- **T1\_1 (Active – Green):** Vehicles are allowed to move straight to Thapathali Road and turn right to Maternity Road.
- **T1\_0 (Partially Active):** Only selected straight movements are allowed to Kuponhole Road and left movements to Maternity Road. The light green indicates it must wait for

vehicle flow to Maternity Road if present, while right-turning movement is restricted to avoid crossing conflicts with T1\_1 flows.

- **T1\_2 (Inactive – Red):** All movements are stopped via signal phase T1\_2.

### **Phase P1**

Phase P1 provides **full priority to T1\_0**, allowing all directional movements, while completely restricting T1\_1 and T1\_2.

- **T1\_0 (Fully Active – Green):** Vehicles can perform through movements and right turns freely to Maternity Road and Kupondole Road, respectively. This enables traffic from Thapathali Road to distribute in all directions, including toward Tripura Marg, Maternity Road, and Kupondole Road.
- **T1\_1 (Inactive – Red):** All movements are stopped via Signal Phase T1\_1.
- **T1\_2 (Inactive – Red):** All movements are stopped via Signal Phase T1\_2.

### **Phase P2**

Phase P2 shifts priority to **T1\_2**, allowing its movements while restricting others.

- **T1\_2 (Active – Green):** Vehicles are allowed to move straight to Maternity Road and turn right to Kupondole Road.
- **T1\_0 (Partially Active):** Some right turn movements to Maternity Road are permitted depending on conflict-free conditions, while others are restricted.
- **T1\_1 (Inactive – Red):** All movements are stopped via Signal phase T1\_1.

### **Traffic Signal Control Strategies**

In the fixed-time control approach, the signal operates on a constant cycle length of 60 seconds, where the phases P0, P1, and P2 are activated sequentially with predefined time allocations. This method does not consider real-time traffic conditions and follows a repetitive cycle regardless of traffic demand, making it simple to implement but less efficient under varying traffic conditions.

In contrast, the DQN-based control approach introduces adaptive signal operation. In this method, the system evaluates traffic conditions at regular intervals of 15 seconds and decides whether to maintain the current phase or switch to another phase based on the observed traffic state. This allows the signal timing to adjust dynamically according to traffic demand, helping to reduce vehicle delays, queue lengths, and overall congestion at the intersection.

To simulate downstream backspill, a random-timer traffic signal with durations of 30, 45, or 60 seconds is placed at the end of each outgoing route to restrict vehicle movement.

The signal enforces a 3-second yellow phase whenever the movement changes from green to red, which is deducted from the total green time. Meanwhile, the main (or Maternity) road operates on a fixed-timer signal with a 45-second green and 15-second red in a cyclic pattern in both fixed-timer and DQN-based modes.

### **5.5. DQN Agent Implementation**

The neural network processed 21-dimensional state inputs through three hidden layers: 128 neurons in the first layer, 64 neurons in the second layer, and 32 neurons in the third layer, all using ReLU activation. The output layer produced 2 Q-values, one for each possible action - STAY and NEXT.

Two networks were maintained: a Q-network for action selection that updated continuously during training, and a target network for stable learning targets that updated every 10 episodes by copying the Q-network weights directly. Experience replay stored up to 10,000 transitions in a circular buffer and randomly sampled mini-batches of 64 transitions for each training step, breaking temporal correlations between consecutive experiences and improving learning stability.

$\epsilon$ -greedy exploration started with fully random actions ( $\epsilon = 1.0$ ) and gradually shifted to exploiting learned Q-values as  $\epsilon$  decayed by a factor of 0.985 per episode, reaching a minimum of  $\epsilon = 0.01$  by approximately episode 300. Q-learning updates computed current Q-values from the Q-network and target Q-values using the Bellman equation - rewards plus discounted maximum future Q-values from the target network with discount factor  $\gamma = 0.95$ . Mean squared error between the current and target Q-values was minimised during training using the Adam optimiser with a learning rate of 0.001.

### **5.6. Training Pipeline Implementation**

Training ran for 300 episodes. Each episode followed this sequence: reset simulation, observe initial state, select action using  $\epsilon$ -greedy policy, apply 3-second yellow phase when switching to next phase, execute 15-second green phase, collect reward averaged over the action duration, observe next state, store transition experience in replay buffer, train on random mini-batch of 64 if buffer contained sufficient samples, and update state for the next decision step.

Between episodes,  $\epsilon$  decayed by multiplying by 0.985, gradually reducing random exploration in favour of learned policy exploitation. Every 10 episodes the target network updated its weights by copying directly from the Q-network, stabilising the learning targets. Model checkpoints were saved every 5 episodes to allow recovery and evaluation at intermediate training stages. Outputs included periodic checkpoints, a final trained

model saved upon completion of all 300 episodes, and training visualisations showing reward, loss, average waiting time, throughput, and epsilon decay across episodes.

### 5.7. Configuration Management

A central configuration file managed all parameters: training settings (episodes = 300, learning rate = 0.001, batch size = 64), DQN hyperparameters ( $\epsilon$  decay = 0.985, discount factor  $\gamma = 0.95$ , replay buffer size = 10,000, target network update frequency = 10 episodes), traffic light identifiers and phase definitions for P1, P2 and P3, lane definitions for origin, stopline and outgoing lanes, PCU values for each vehicle type present at Thapathali T1, reward weighting factor  $\lambda = 0.0004$ , SUMO simulation file paths, and action durations (yellow transition: 3 seconds, green phase: 15 seconds per decision step). Centralising all parameters in a single configuration file ensured consistency across training, evaluation and comparison scripts and allowed hyperparameter adjustments without modifying the core environment or agent code.

### 5.8. Metric Calculation

Three primary metrics were tracked throughout training and evaluation to assess intersection performance.

Average Waiting Time was calculated by collecting the waiting time of all vehicles present on the incoming route lanes at regular intervals throughout each episode. At each metric collection step, the mean waiting time across all observed vehicles was accumulated and divided by the total number of collection steps to produce the episode-level average waiting time in seconds.

Average Queue Length was calculated by recording the number of halting vehicles across all incoming route lanes at each metric collection step. A vehicle was considered halting if it was either fully stopped below the SUMO default threshold of 0.1 m/s or slow-crawling below 1.5 m/s. The mean halting count across all monitored lanes was accumulated and divided by the total number of collection steps to produce the episode-level average queue length in vehicles.

Throughput per Wait Second was calculated as a composite efficiency metric by dividing the total episode throughput by the average waiting time, quantifying how many vehicles were processed per second of average delay experienced at the intersection:

$$\text{Throughput per Wait Second} = \frac{\text{Total Throughput (vehicles)}}{\text{Average Waiting Time (seconds)}}$$



A higher value indicates a more efficient intersection operation, with more vehicles cleared per unit of delay experienced, allowing direct comparison between the DQN agent and the fixed timer baseline across different traffic conditions.

## 6. RESULTS AND ANALYSIS

### 6.1. Simulation Environment Configuration

The DQN-based traffic signal control system was implemented using a realistic simulation of the Thapathali intersection in SUMO. The environment was constructed using OpenStreetMap data and calibrated with OD patterns collected from field studies. As shown in Figure 6-1, the simulation represents the four major approaches: Maitighar, Kupondole, Tripureshwor, and Maternity, with proper lane configurations and traffic signal placements. The simulation incorporated diverse vehicle types using PCU factors from empirical studies, and traffic demand was generated using the allocation algorithms detailed in Chapter 5. The DQN agent underwent training over 300 episodes with an exploration rate decaying from 1.0 to 0.01, utilizing the Max-Pressure reward function to optimize congestion management.

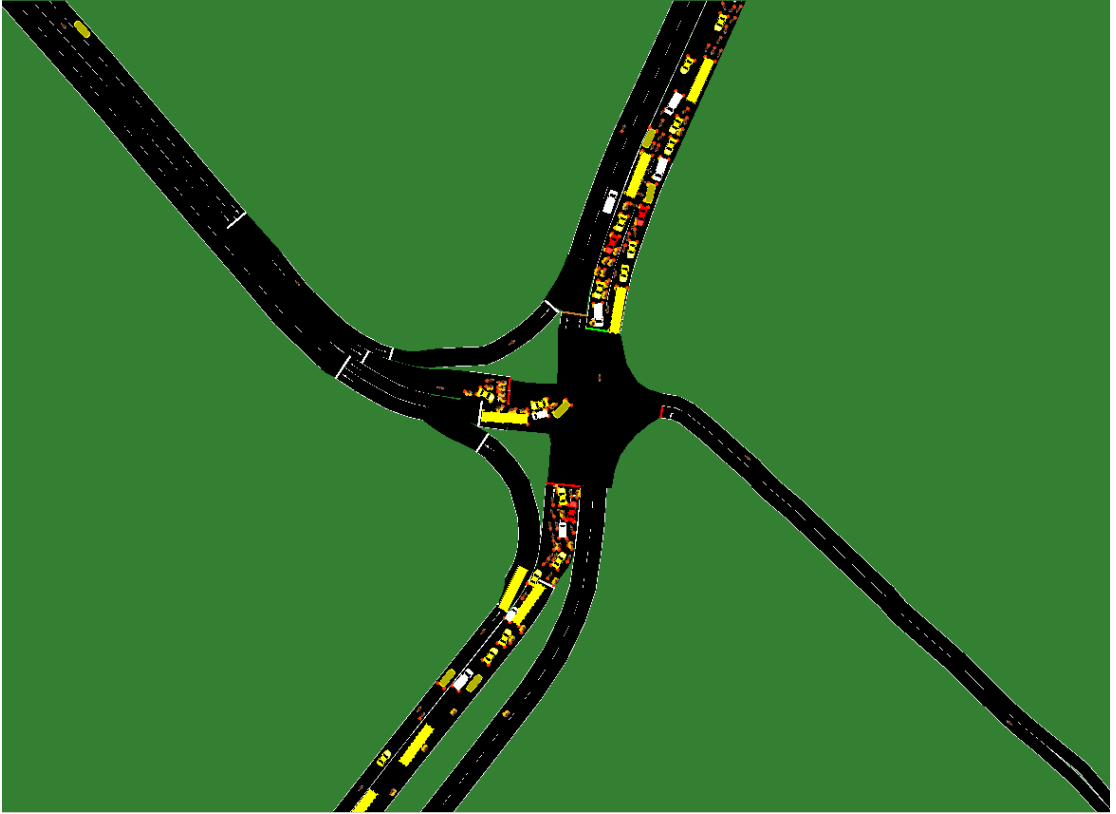


Figure 6-1. SUMO Simulation Environment of Thapathali Intersection

### 6.2. Training Convergence and Learning Trajectory

Figure 6-2 shows the episode reward and epsilon decay curve over 300 training episodes. During early exploration (episodes 0–50), rewards were highly variable at around ~5,500 to ~6,000. A sharp improvement began after episode 50 as epsilon decayed and exploitation increased. The 10-episode moving average reward rose steadily from

approximately ~-5,300 to -3,395.8 by episode 300, a ~43% net improvement. The upward trend throughout training confirms successful policy convergence, with the agent transitioning fully to exploitation by episode 300.

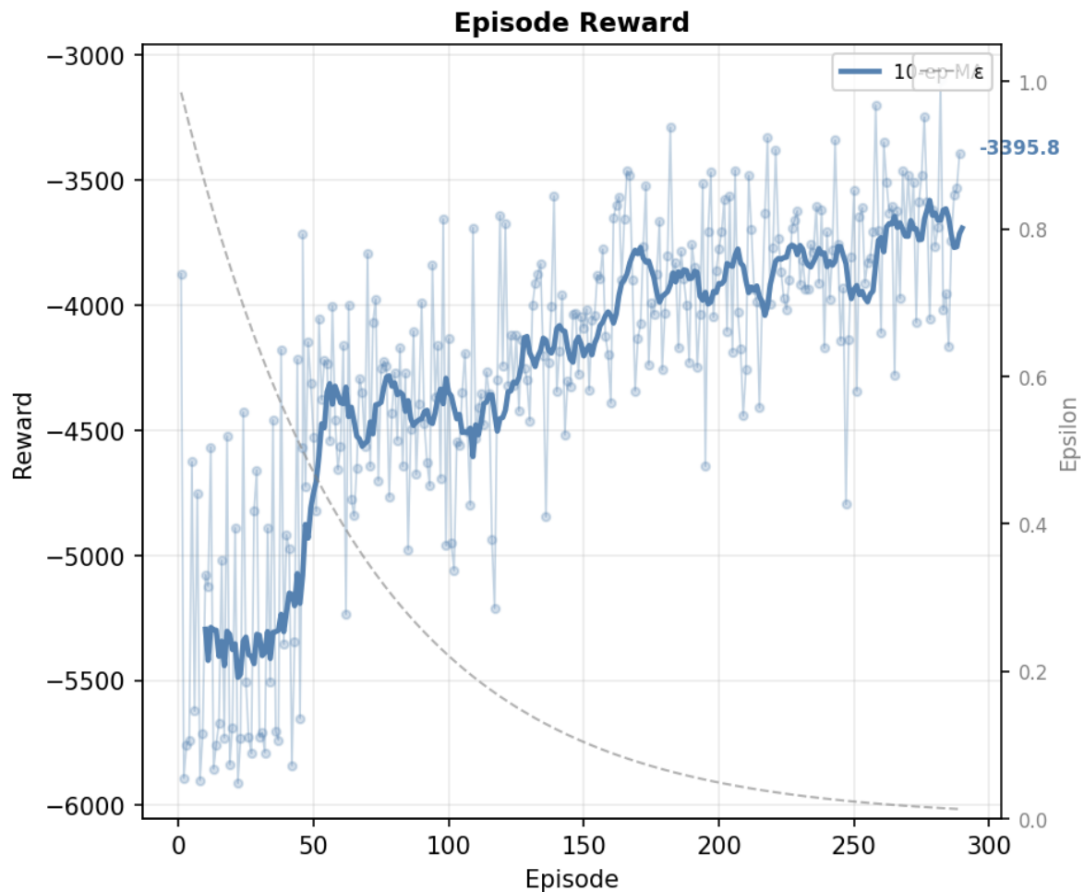


Figure 6-2. Episode Reward and Epsilon Decay over 300 Training Episodes

### 6.3. Wait Time Reduction Analysis

Figure 6-3 compares hourly average wait times for the Fixed 60s controller and the DQN agent (Episode 300) on Day 5. The fixed controller ranges from 29s to 50s with high volatility, while the DQN consistently stays between 23s and 30s.

Key results:

- Overall average: **+22.90%** improvement across all hours
- Morning peak average (9–11 AM): **+8.26%** improvement
- Evening Peak Average (4–6 PM): **+19.26%** improvement

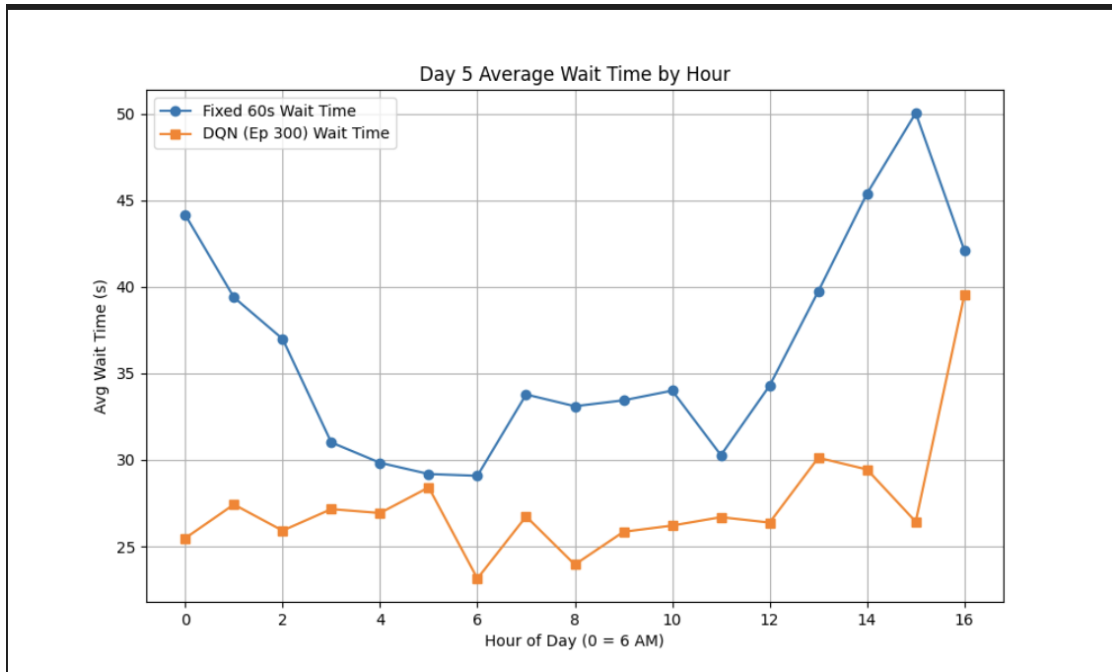


Figure 6-3. Day 5 Average Wait Time by Hour: Fixed 60s vs DQN (Episode 300)

#### 6.4. Queue Length Management Effectiveness

Figure 6-4 shows hourly average total queue length. The fixed controller sustains 10–13 vehicles throughout peak hours, while the DQN holds queues to 5.9–10.3 vehicles. In the evening, the DQN clears queues to 1.83 vehicles by 22:00 versus 5.0 for the fixed plan.

Key results:

- Overall average: **+30.06%** reduction across all hours
- Morning peak average (9–11 AM): **+21.42%** improvement
- Evening Peak Average (4–6 PM): **+20.89%** improvement

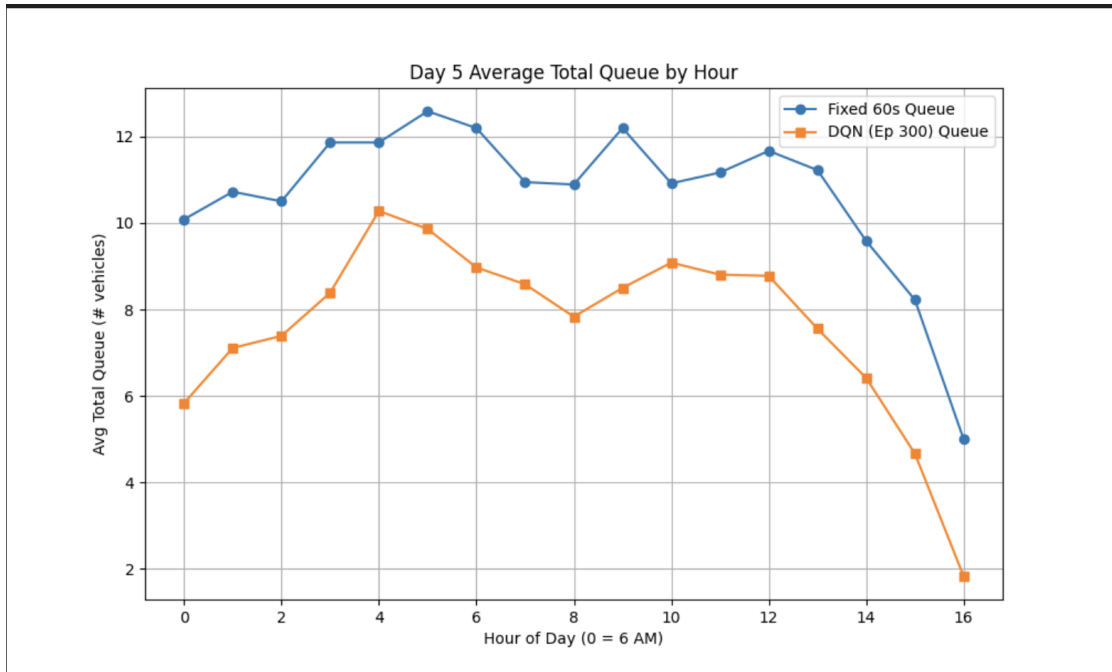


Figure 6-4. Day 5 Average Total Queue Length by Hour: Fixed 60s vs DQN (Episode 300)

## 6.5. Throughput Efficiency Analysis

Figure 6-5 shows the throughput efficiency ratio (vehicles per wait-second). The DQN curve consistently exceeds the F60 curve at all hours, with the gap widening during peak and off-peak extremes.

Key results:

- Overall average: **+32.65%** reduction across all hours
- Morning peak average (9–11 AM): **+9.33%** improvement
- Evening Peak Average (4–6 PM): **+24.23%** improvement

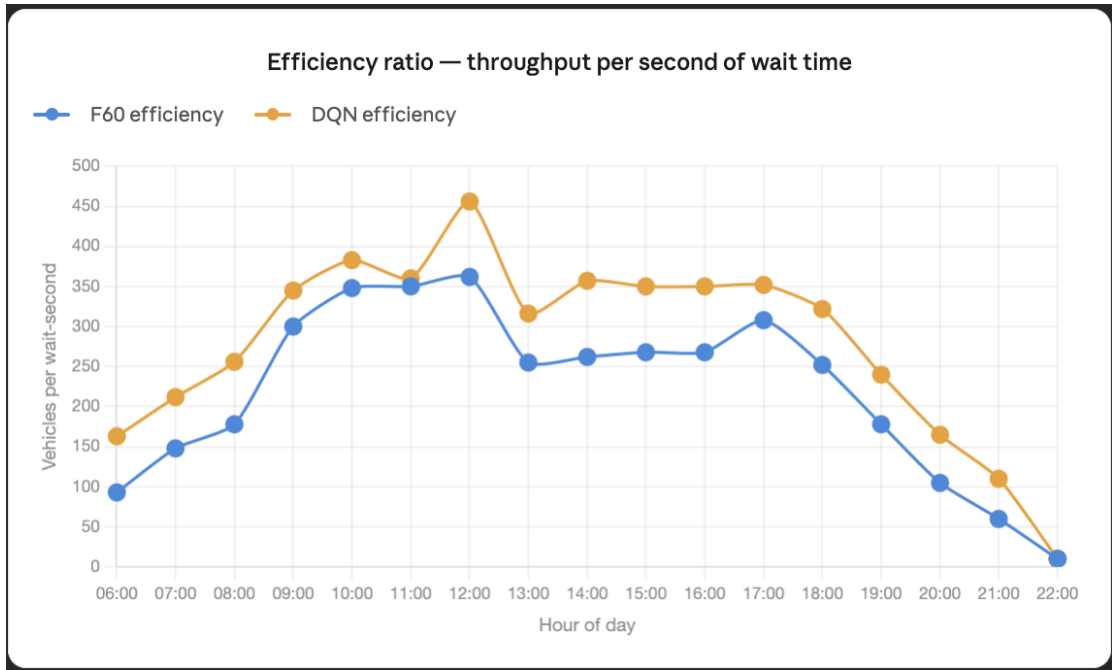


Figure 6-5. Efficiency Ratio of Throughput per Second of Wait Time: F60 vs DQN (Episode 300)

## 6.6. Consolidated Performance Comparison

Table 6.6.1 summarises all three performance metrics hour by hour. The DQN agent outperforms the fixed-time controller across every hour and every metric.

Table 6.6.1. Hour-by-Hour Performance: Wait Time and Queue Length (Episode 300)

| Time                                                                 | Wait time (s)<br>Fixed | Wait time (s)<br>Imp. Queue | Improvement<br>(%) | Queue length (veh)<br>Fixed | Queue length (veh)<br>Imp. Queue |
|----------------------------------------------------------------------|------------------------|-----------------------------|--------------------|-----------------------------|----------------------------------|
| 06:00                                                                | 44.16                  | 25.48                       | 42.30%             | 10.08                       | 5.83                             |
| 07:00                                                                | 39.39                  | 27.43                       | 30.37%             | 10.72                       | 7.11                             |
| 08:00                                                                | 36.99                  | 25.92                       | 29.93%             | 10.50                       | 7.39                             |
| 09:00                                                                | 31.01                  | 27.16                       | 12.42%             | 11.86                       | 8.39                             |
| 10:00                                                                | 29.84                  | 26.93                       | 9.73%              | 11.86                       | 10.28                            |
| 11:00                                                                | 29.19                  | 28.41                       | 2.65%              | 12.58                       | 9.86                             |
| 12:00                                                                | 29.08                  | 23.15                       | 20.40%             | 12.19                       | 8.97                             |
| 13:00                                                                | 33.78                  | 26.74                       | 20.85%             | 10.94                       | 8.58                             |
| 14:00                                                                | 33.10                  | 23.96                       | 27.63%             | 10.89                       | 7.83                             |
| 15:00                                                                | 33.44                  | 25.85                       | 22.70%             | 12.19                       | 8.50                             |
| 16:00                                                                | 34.00                  | 26.21                       | 22.92%             | 10.92                       | 9.08                             |
| 17:00                                                                | 30.27                  | 26.69                       | 11.82%             | 11.17                       | 8.81                             |
| 18:00                                                                | 34.29                  | 26.38                       | 23.06%             | 11.67                       | 8.78                             |
| 19:00                                                                | 39.74                  | 30.13                       | 24.18%             | 11.22                       | 7.56                             |
| 20:00                                                                | 45.38                  | 29.45                       | 35.11%             | 9.58                        | 6.42                             |
| 21:00                                                                | 50.03                  | 26.43                       | 47.17%             | 8.22                        | 4.67                             |
| 22:00                                                                | 42.09                  | 39.53                       | 6.10%              | 5.00                        | 1.83                             |
| Overall Average Improvement: Wait Time +22.90%, Queue Length +30.06% |                        |                             |                    |                             |                                  |

Table 6.6.2. Hour-by-Hour Performance: Throughput Efficiency (Episode 300)

| Time                                        | Throughput efficiency (veh/wait-s)<br>Fixed | Throughput efficiency (veh/wait-s)<br>Imp. Queue |
|---------------------------------------------|---------------------------------------------|--------------------------------------------------|
| 06:00                                       | 93                                          | 163                                              |
| 07:00                                       | 148                                         | 212                                              |
| 08:00                                       | 178                                         | 256                                              |
| 09:00                                       | 300                                         | 345                                              |
| 10:00                                       | 348                                         | 383                                              |
| 11:00                                       | 350                                         | 360                                              |
| 12:00                                       | 362                                         | 456                                              |
| 13:00                                       | 255                                         | 316                                              |
| 14:00                                       | 262                                         | 357                                              |
| 15:00                                       | 268                                         | 350                                              |
| 16:00                                       | 268                                         | 350                                              |
| 17:00                                       | 308                                         | 352                                              |
| 18:00                                       | 252                                         | 322                                              |
| 19:00                                       | 178                                         | 240                                              |
| 20:00                                       | 105                                         | 165                                              |
| 21:00                                       | 60                                          | 110                                              |
| 22:00                                       | 10                                          | 10                                               |
| <i>Overall Average Improvement: +32.65%</i> |                                             |                                                  |

## 6.7. Overall Performance Summary and Analysis

The three metrics evaluated - average wait time, average queue length, and throughput efficiency ratio - collectively present a consistent and unambiguous picture: the DQN agent outperforms the fixed 60-second controller across every hour of the simulation day and across every dimension of traffic performance.

### 6.7.1. Consistency Across Metrics

The convergence of all three metrics in favour of the DQN policy is analytically significant. A controller that reduces wait time by redistributing green time aggressively toward one approach could, in principle, inflate queue lengths elsewhere. The simultaneous 22.90% reduction in wait time, 30.06% reduction in queue length, and 32.65% improvement in throughput efficiency ratio confirm that no such trade-off occurred - the agent achieved system-wide improvement rather than localised optimisation at the cost of other movements.

### 6.7.2. Throughput Efficiency Ratio Analysis

As shown in Figure 6-5, the throughput efficiency ratio measured as vehicles processed per second of cumulative wait time favoured the DQN agent at every hour of the day, with the gap widening notably during peak and late off-peak periods. The overall 32.65% improvement is the largest gain observed across all three metrics, which is particularly

meaningful given that total vehicle throughput remained comparable between the two controllers. This confirms that the DQN agent did not improve efficiency by processing more vehicles but rather by moving the same number of vehicles through the intersection with substantially less cumulative delay. During the morning peak (9-11 AM) the efficiency ratio improved by 9.33%, and during the evening peak (4-6 PM) by 24.23%, mirroring the pattern observed in wait time and queue length. The widening gap during evening peak hours suggests that the DQN policy is most differentiated from the fixed plan precisely when demand is heaviest and signal timing decisions carry the greatest consequence.

### **6.7.3. Peak Hour Behaviour**

During the morning peak (9-11 AM), improvements were relatively moderate across all metrics: 8.26% in wait time, 21.42% in queue length, and 9.33% in throughput efficiency. In contrast, the evening peak (4-6 PM) yielded 19.26%, 20.89%, and 24.23% improvements respectively. This asymmetry is consistent with the nature of the Max-Pressure reward function - the agent responds proportionally to pressure differentials across the four approaches. The evening peak at Thapathali tends to produce more uneven directional demand, giving the adaptive policy greater opportunity to outperform the fixed plan's uniform phase allocation. The morning peak, by contrast, produces more symmetric saturation across all approaches simultaneously, leaving less room for adaptive improvement over a well-calibrated fixed plan.

### **6.7.4. Variance Reduction**

The fixed controller's wait time ranged from 29s to 50s across hours while the DQN agent remained consistently within 23-30s. Similarly, queue lengths under the fixed plan sustained 10-13 vehicles during peak hours versus 5.9-10.3 under DQN. The throughput efficiency curve in Figure 6-5 reflects this same stability - the DQN ratio curve is not only higher but smoother than the F60 curve, indicating that the agent maintained consistent processing efficiency rather than producing sporadic bursts of high throughput separated by periods of congestion. This variance reduction is practically significant because it means commuters experience more predictable and uniform delay rather than the wide swings characteristic of fixed-time control.

### **6.7.5. Off-Peak Performance**

The DQN agent's advantage extends clearly into off-peak hours, with queue length falling to 1.83 vehicles by 22:00 compared to 5.0 under the fixed plan, and the throughput efficiency gap remaining visible even at low demand. This reflects the agent's ability to shorten unnecessary red phases when vehicle arrivals are sparse - a structural limitation



of fixed-time control that allocates green time uniformly regardless of actual demand, thereby accumulating wait time even when queues are minimal. The 47.17% wait time improvement at 21:00 is the single largest hourly gain recorded, highlighting precisely how wasteful fixed-cycle operation becomes during low-demand periods.

#### 6.7.6. Failure Case Analysis

While the DQN agent demonstrated consistent improvement over the fixed timer across all hours and metrics, several failure cases and structural limitations were identified that qualify the scope of these results:

- **Peak-hour saturation:** The most significant failure case occurs during 10:00-11:00 where the wait time improvement falls to just 2.65%, indicating that the agent's adaptive advantage essentially disappears when all approaches are simultaneously overloaded. This is not an algorithmic failure but an infrastructural constraint - the Thapathali intersection operates beyond its designed capacity during peak windows and no signal timing strategy alone can resolve that.
- **Reward signal degradation during saturation:** The Max-Pressure reward compares stopline PCU against outgoing PCU but stopline lanes at Thapathali are short and fill almost instantly during peak hours, making the pressure signal nearly constant regardless of what the agent does. This reduces the quality of the learning signal precisely during the periods where better control would matter most.
- **Absence of real downstream network modelling:** Random timer signals were placed at outgoing lane ends to simulate backspill but these were not calibrated to actual neighbouring intersection behaviour, meaning the agent may have learned a policy optimised for an artificial downstream condition that does not fully reflect reality.
- **Exclusion of non-motorised and multi-modal traffic:** The simulation excludes pedestrians, public transport, and non-motorised vehicles which are present at the real Thapathali intersection. Their omission means the agent operates in a cleaner environment than would exist in actual deployment, and real-world performance could differ meaningfully from simulation results.
- **Limited training data diversity:** The training was conducted exclusively on four days of observed traffic data, limiting the diversity of demand patterns the agent was exposed to. Unusual traffic conditions such as accidents, festivals, road closures, or sudden demand surges were not represented in training, and the agent's ability to generalise to such scenarios remains untested.

### **6.7.7. Summary**

Taken together, these results confirm that the DQN policy learned a generalisable and robust signal control strategy, not merely one optimised for a narrow demand condition. The consistency of improvement across wait time, queue length, and throughput efficiency and the alignment of their peak-hour patterns indicate that all three metrics are capturing the same underlying phenomenon-the DQN agent allocates green time more precisely in response to real-time intersection state, reducing both the duration and the downstream consequences of vehicle delay. The hour-by-hour breakdown in Table 6.6.1 supports this interpretation, showing no hour in which the fixed controller equals or surpasses the DQN on any metric.

## **7. CONCLUSION**

### **7.1. Achievements**

The following Achievements were achieved in this project:

- Data collection and vehicle allocation from relevant sources.
- Configuration of SUMO for the Thapathali intersection, including road layout, traffic flow, and signal configuration.
- Setup of the simulation environment for testing the traffic control system.
- Fixed-timer logic configuration for baseline comparison.
- DQN agent design and implementation for traffic signal optimisation.
- The average waiting time over the 16-hour period (6 AM to 10 PM) was reduced by 22.90%.
- The average queue length over the 16-hour period was reduced by 30.06%.
- The average throughput per waiting second over the 16-hour period improved by 32.65%.

### **7.2. Limitations**

Some factors were not considered or implemented in the current system; these are outlined below as limitations.

- No hardware implementation or real-world field testing with traffic signals.
- Pedestrians and public transport are excluded in this phase to keep the control problem tractable and focus on vehicle-delay reduction at Thapathali; modeling them requires additional constraints and reliable data.
- Extreme scenarios or multi-modal integration, like ride-sharing or parking systems, are not considered.
- Random traffic signal timers were placed at the end of outgoing lanes to simulate network backspill; however, these were not based on the actual traffic conditions of the neighbouring network

### **7.3. Future Work**

Future development may include:

- Multi-Intersection Coordination: Scaling the system to manage multiple connected intersections.

- **Faster Training Using Alternative Simulators:** The current training pipeline relies on SUMO, which, while realistic, is computationally intensive and limits the speed at which the agent can be trained across large numbers of episodes. Future work could explore lightweight traffic simulation platforms such as CityFlow, which is specifically designed for large-scale reinforcement learning research and can simulate traffic significantly faster than SUMO by sacrificing some physical realism.
- **Advanced Optimization:** Exploring hybrid algorithms or multi-agent RL for more

complex network topologies.

#### **7.4. Final Remarks**

This project set out to investigate whether a reinforcement learning-based approach could meaningfully improve traffic signal control at the Thapathali intersection in Kathmandu, one of the city's most persistently congested arterial junctions. The results suggest that it can, and the case for adaptive signal control is stronger than the numbers alone convey.

The DQN agent demonstrated consistent improvement over fixed-timer logic across the full operational window. Where the gains are most evident is during low-to-moderate demand periods, hours where fixed timing is structurally wasteful, holding vehicles at red signals long after opposing flows have cleared. In these windows, the agent's ability to read and react to real-time queue states yielded noticeably better outcomes, highlighting just how rigid and inefficient schedule-based control becomes when traffic is light and variable.

The more telling story, however, lies in the peak hours. Improvements during high-demand periods, the morning and evening surges that define Kathmandu's daily congestion profile, were comparatively modest. This is not a failure of the algorithm; it is a reflection of a harder problem. At genuine saturation, when every approach is simultaneously overloaded, no signal timing strategy alone can conjure capacity that does not exist. The Thapathali intersection, like many in the Kathmandu valley, operates well beyond its designed throughput during peak windows. The physical geometry, lane discipline, and competing turning movements create a level of congestion that signal optimisation can soften but cannot resolve. The DQN agent improves the allocation of insufficient green time; it does not manufacture more road space.

This distinction matters for how the results should be interpreted. The system performs well precisely where fixed timers fail most obviously, and it reaches its natural ceiling precisely where the underlying infrastructure is the binding constraint. That ceiling is not an algorithmic limitation but an infrastructural one, and it points toward what

complementary interventions would be needed alongside smarter signal control to meaningfully address peak-hour gridlock.

As a proof of concept, this work establishes a credible and reproducible foundation. The simulation pipeline, agent design, and evaluation framework are all in place. What lies ahead, including real-world hardware integration, inclusion of pedestrians and public transport, and coordination across multiple connected intersections, are well-defined extensions rather than open questions, each building directly on what has been demonstrated here.

Urban traffic management in cities like Kathmandu has long depended on static, schedule-driven systems that cannot adapt to the variability and intensity of real-world demand. This project offers substantive early evidence that adaptive, learning-based control can meaningfully close that gap, not as a complete solution, but as an important and scalable component of one.

## A. CODE SNIPPETS

### A.1. DQN Network Architecture

The following snippet defines the architecture of the DQN model used for traffic signal control.

```
1 class DQNetwork(nn.Module):
2     """
3     Deep Q-Network.
4     Input : 21-dim state [7 origin PCU | 6 stopline wait | 5 outgoing PCU | 3 one-hot]
5     Output: Q-value for STAY (0) and NEXT (1)
6     Architecture: 128 -> 64 -> 32 hidden layers, ReLU activations
7     """
8
9     def __init__(self, state_size, action_size):
10         super(DQNetwork, self).__init__()
11         layers = []
12         in_size = state_size
13         for out_size in config.HIDDEN_LAYERS:
14             layers.append(nn.Linear(in_size, out_size))
15             layers.append(nn.ReLU())
16             in_size = out_size
17         layers.append(nn.Linear(in_size, action_size))
18         self.net = nn.Sequential(*layers)
19
20     def forward(self, x):
21         return self.net(x)
```

Listing A.1 DQN Model Architecture

### A.2. Replay Buffer

This snippet demonstrates how experiences (state, action, reward, next state, done) are stored and sampled in the replay buffer.

```
1 class ReplayBuffer:
2     """
3     Circular experience replay memory -- capacity 10,000.
4     Stores (state, action, reward, next_state, done).
5     Random sampling breaks time correlation so agent learns
6     from mix of all traffic patterns simultaneously.
7     """
8
9     def __init__(self, capacity):
10         self.buffer = deque(maxlen=capacity)
11
12     def add(self, state, action, reward, next_state, done):
13         self.buffer.append((state, action, reward, next_state, done))
14
15     def sample(self, batch_size):
16         batch = random.sample(self.buffer, batch_size)
17         s, a, r, ns, d = zip(*batch)
18         return (
19             np.array(s, dtype=np.float32),
20             np.array(a, dtype=np.int64),
```

```

21         np.array(r, dtype=np.float32),
22         np.array(ns, dtype=np.float32),
23         np.array(d, dtype=np.float32),
24     )
25
26     def __len__(self):
27         return len(self.buffer)

```

Listing A.2 Replay Buffer Implementation

### A.3. DQN Agent

This snippet defines the DQN agent that interacts with the traffic control environment, selecting actions and updating the model.

```

1  class DQNAgent:
2
3      def __init__(self, state_size=config.STATE_SIZE,
4                    action_size=config.NUM_ACTIONS):
5          self.state_size = state_size
6          self.action_size = action_size
7
8          self.gamma = config.GAMMA
9          self.epsilon = config.EPSILON_START
10         self.epsilon_min = config.EPSILON_MIN
11         self.epsilon_decay = config.EPSILON_DECAY
12         self.batch_size = config.BATCH_SIZE
13
14         # Device selection: MPS (Apple Metal GPU) > CUDA > CPU
15         if torch.backends.mps.is_available():
16             self.device = torch.device("mps")
17         elif torch.cuda.is_available():
18             self.device = torch.device("cuda")
19         else:
20             self.device = torch.device("cpu")
21         print(f" [DQN] Using device: {self.device}")
22
23         self.q_net = DQNetwork(state_size, action_size).to(self.device)
24         self.target_net = DQNetwork(state_size, action_size).to(self.device)
25         self.sync_target()
26
27         self.optimizer = optim.Adam(
28             self.q_net.parameters(), lr=config.LEARNING_RATE)
29         self.loss_fn = nn.MSELoss()
30
31         self.memory = ReplayBuffer(config.REPLAY_BUFFER_SIZE)
32         self.total_steps = 0
33
34     def sync_target(self):
35         self.target_net.load_state_dict(self.q_net.state_dict())
36
37     def act(self, state, training=True):
38
39         if training and random.random() < self.epsilon:
40             return random.randrange(self.action_size)
41         with torch.no_grad():

```

```

42         t = torch.FloatTensor(state).unsqueeze(0).to(self.device)
43         return self.q_net(t).argmax().item()

```

Listing A.3 DQN Agent Implementation

## A.4. Model Training

This snippet demonstrates the process of training the DQN agent, including experience replay and the loss calculation.

```

1 def train_step(self):
2     """
3     Train on one random batch.
4     Bellman target: reward + gamma * max(target_net(next_state))
5     Returns loss value, 0.0 if buffer not full yet.
6     """
7     if len(self.memory) < self.batch_size:
8         return 0.0
9
10    states, actions, rewards, next_states, dones = \
11        self.memory.sample(self.batch_size)
12
13    states = torch.FloatTensor(states).to(self.device)
14    actions = torch.LongTensor(actions).to(self.device)
15    rewards = torch.FloatTensor(rewards).to(self.device)
16    next_states = torch.FloatTensor(next_states).to(self.device)
17    dones = torch.FloatTensor(dones).to(self.device)
18
19    current_q = self.q_net(states).gather(
20        1, actions.unsqueeze(1)).squeeze(1)
21
22    with torch.no_grad():
23        next_q = self.target_net(next_states).max(1)[0]
24        target_q = rewards + (1 - dones) * self.gamma * next_q
25
26    loss = self.loss_fn(current_q, target_q)
27    self.optimizer.zero_grad()
28    loss.backward()
29    torch.nn.utils.clip_grad_norm_(self.q_net.parameters(), 1.0)
30    self.optimizer.step()
31
32    self.total_steps += 1
33    return loss.item()

```

Listing A.4 Training the DQN Agent

## A.5. Model Saving and Loading

The following snippet shows how to save and load the model state.

```

1 def save(self, path):
2     torch.save({
3         'q_net': self.q_net.state_dict(),
4         'target_net': self.target_net.state_dict(),
5         'optimizer': self.optimizer.state_dict(),

```



```

6         'epsilon':      self.epsilon,
7         'total_steps': self.total_steps,
8     }, path)
9     print(f"    [DQN] Saved -> {path}")
10
11     def load(self, path):
12         ckpt = torch.load(path, map_location=self.device)
13         self.q_net.load_state_dict(ckpt['q_net'])
14         self.target_net.load_state_dict(ckpt['target_net'])
15         self.optimizer.load_state_dict(ckpt['optimizer'])
16         self.epsilon      = ckpt['epsilon']
17         self.total_steps = ckpt.get('total_steps', 0)
18         print(f"    [DQN] Loaded <- {path}    (epsilon={self.epsilon:.4f})")

```

Listing A.5 Model Saving and Loading

## A.6. Summary

The presented code snippets demonstrate the practical implementation of the DQN-based traffic control system. These snippets highlight how reinforcement learning and neural networks are applied to optimize traffic flow at the Thapathali intersection.

## B. PROJECT BUDGET

This project is designed to be completed with minimal cost by relying on open-source tools (e.g., SUMO) and free software resources. The DQN development and training will be performed using available local computing resources. If additional compute is needed, we will use free-tier subscriptions, academic/free credits (if available), or support from friends/peers with suitable hardware instead of paid cloud services. Therefore, the budget mainly covers documentation and small miscellaneous expenses.

Table B.0.1. Project Budget Allocation

| Task                                          | Cost (NRP)   | Total (NRP)     |
|-----------------------------------------------|--------------|-----------------|
| Documentation (printing, binding, stationery) | RS. 1200     | RS. 1200        |
| Miscellaneous                                 | RS. 1000     | RS. 1000        |
|                                               | <b>Total</b> | <b>RS. 2200</b> |

C. PROJECT TIMELINE

The project will be organized into distinct phases, each focusing on specific tasks with well-defined milestones. These phases will ensure structured and efficient progress, with regular evaluations to monitor the advancement toward successful project completion.

C.1. Gantt Chart

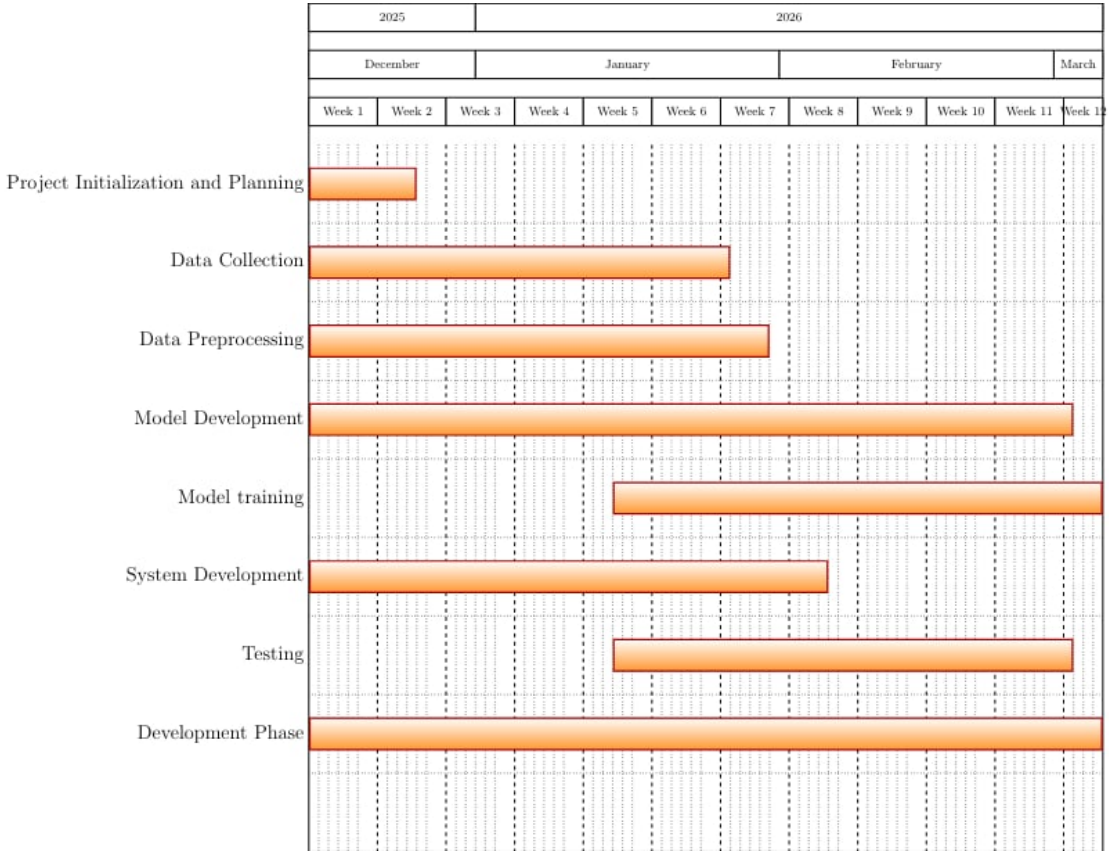


Figure C-1. Project Gantt Chart

## D. SUMMARY OF DATA

This dataset includes portions of vehicle counts alongside PCU OD data:

Table D.0.1. 15-Minute Interval Vehicle Counts by Class: Day 1 (06:00–15:15)

| Start Time | End Time | HV | LV | B  | MB | MIB | C   | U  | F   | T  | TW   |
|------------|----------|----|----|----|----|-----|-----|----|-----|----|------|
| 06:00:00   | 06:15:00 | 0  | 3  | 19 | 4  | 23  | 107 | 26 | 9   | 18 | 545  |
| 06:15:00   | 06:30:00 | 0  | 4  | 15 | 13 | 34  | 166 | 38 | 14  | 22 | 780  |
| 06:30:00   | 06:45:00 | 3  | 8  | 24 | 12 | 35  | 153 | 38 | 24  | 28 | 828  |
| 06:45:00   | 07:00:00 | 3  | 3  | 34 | 9  | 34  | 189 | 35 | 37  | 30 | 963  |
| 07:00:00   | 07:15:00 | 1  | 5  | 28 | 9  | 35  | 207 | 38 | 32  | 21 | 956  |
| 07:15:00   | 07:30:00 | 2  | 4  | 29 | 14 | 45  | 229 | 36 | 42  | 24 | 993  |
| 07:30:00   | 07:45:00 | 0  | 7  | 28 | 12 | 31  | 304 | 45 | 58  | 27 | 1093 |
| 07:45:00   | 08:00:00 | 4  | 4  | 32 | 9  | 39  | 270 | 38 | 48  | 26 | 1102 |
| 08:00:00   | 08:15:00 | 0  | 6  | 18 | 13 | 38  | 265 | 46 | 53  | 31 | 1048 |
| 08:15:00   | 08:30:00 | 2  | 7  | 34 | 7  | 33  | 324 | 34 | 58  | 26 | 1093 |
| 08:30:00   | 08:45:00 | 1  | 4  | 36 | 12 | 37  | 333 | 39 | 53  | 25 | 1137 |
| 08:45:00   | 09:00:00 | 1  | 5  | 27 | 7  | 44  | 368 | 43 | 66  | 29 | 1332 |
| 09:00:00   | 09:15:00 | 0  | 3  | 23 | 9  | 39  | 356 | 33 | 73  | 34 | 1434 |
| 09:15:00   | 09:30:00 | 2  | 6  | 21 | 14 | 33  | 376 | 27 | 84  | 35 | 1689 |
| 09:30:00   | 09:45:00 | 0  | 4  | 26 | 10 | 39  | 379 | 30 | 89  | 37 | 1957 |
| 09:45:00   | 10:00:00 | 0  | 3  | 20 | 10 | 40  | 383 | 21 | 78  | 45 | 2117 |
| 10:00:00   | 10:15:00 | 0  | 2  | 20 | 11 | 41  | 421 | 33 | 64  | 37 | 2155 |
| 10:15:00   | 10:30:00 | 0  | 5  | 18 | 13 | 41  | 448 | 33 | 87  | 40 | 2027 |
| 10:30:00   | 10:45:00 | 1  | 3  | 25 | 10 | 41  | 433 | 37 | 76  | 33 | 1824 |
| 10:45:00   | 11:00:00 | 0  | 6  | 21 | 15 | 37  | 445 | 51 | 90  | 33 | 1880 |
| 11:00:00   | 11:15:00 | 0  | 2  | 23 | 8  | 33  | 452 | 43 | 97  | 40 | 1904 |
| 11:15:00   | 11:30:00 | 1  | 6  | 24 | 4  | 39  | 461 | 47 | 95  | 29 | 1840 |
| 11:30:00   | 11:45:00 | 1  | 5  | 21 | 10 | 44  | 455 | 68 | 85  | 36 | 1921 |
| 11:45:00   | 12:00:00 | 1  | 4  | 22 | 9  | 39  | 454 | 64 | 94  | 33 | 1968 |
| 12:00:00   | 12:15:00 | 1  | 3  | 20 | 12 | 40  | 480 | 62 | 105 | 21 | 1970 |
| 12:15:00   | 12:30:00 | 0  | 4  | 24 | 8  | 33  | 435 | 59 | 88  | 26 | 1963 |
| 12:30:00   | 12:45:00 | 0  | 5  | 24 | 9  | 39  | 448 | 73 | 91  | 31 | 2025 |
| 12:45:00   | 13:00:00 | 1  | 3  | 23 | 8  | 34  | 442 | 66 | 107 | 32 | 1953 |
| 13:00:00   | 13:15:00 | 1  | 6  | 21 | 7  | 39  | 447 | 69 | 96  | 24 | 1817 |
| 13:15:00   | 13:30:00 | 1  | 6  | 22 | 10 | 36  | 507 | 69 | 117 | 27 | 1453 |
| 13:30:00   | 13:45:00 | 0  | 7  | 24 | 6  | 38  | 541 | 80 | 117 | 35 | 1289 |
| 13:45:00   | 14:00:00 | 1  | 5  | 24 | 10 | 34  | 471 | 57 | 104 | 24 | 1078 |
| 14:00:00   | 14:15:00 | 1  | 11 | 20 | 9  | 33  | 465 | 41 | 93  | 18 | 1176 |
| 14:15:00   | 14:30:00 | 1  | 7  | 16 | 8  | 36  | 506 | 50 | 112 | 26 | 1392 |
| 14:30:00   | 14:45:00 | 2  | 9  | 23 | 13 | 28  | 499 | 58 | 120 | 28 | 1595 |
| 14:45:00   | 15:00:00 | 0  | 4  | 22 | 6  | 36  | 507 | 65 | 106 | 27 | 1608 |
| 15:00:00   | 15:15:00 | 0  | 6  | 22 | 8  | 29  | 496 | 51 | 131 | 29 | 1647 |

Table D.0.2. 15-Minute Interval Vehicle Counts by Class: Day 1 (15:15–22:15)

| Start Time | End Time | HV | LV | B  | MB | MIB | C   | U  | F   | T  | TW   |
|------------|----------|----|----|----|----|-----|-----|----|-----|----|------|
| 15:15:00   | 15:30:00 | 0  | 1  | 22 | 5  | 39  | 502 | 51 | 93  | 34 | 1572 |
| 15:30:00   | 15:45:00 | 0  | 9  | 20 | 7  | 37  | 478 | 49 | 102 | 26 | 1578 |
| 15:45:00   | 16:00:00 | 1  | 0  | 20 | 13 | 34  | 454 | 54 | 111 | 28 | 1485 |
| 16:00:00   | 16:15:00 | 0  | 5  | 21 | 12 | 34  | 455 | 41 | 107 | 25 | 1536 |
| 16:15:00   | 16:30:00 | 0  | 1  | 19 | 12 | 39  | 477 | 44 | 112 | 20 | 1576 |
| 16:30:00   | 16:45:00 | 1  | 4  | 22 | 8  | 36  | 494 | 45 | 108 | 37 | 1704 |
| 16:45:00   | 17:00:00 | 0  | 3  | 27 | 8  | 35  | 462 | 48 | 140 | 29 | 1621 |
| 17:00:00   | 17:15:00 | 0  | 8  | 17 | 9  | 41  | 442 | 33 | 121 | 22 | 1606 |
| 17:15:00   | 17:30:00 | 0  | 5  | 25 | 5  | 45  | 476 | 32 | 126 | 24 | 1735 |
| 17:30:00   | 17:45:00 | 0  | 2  | 17 | 12 | 37  | 459 | 34 | 122 | 25 | 1716 |
| 17:45:00   | 18:00:00 | 1  | 7  | 24 | 12 | 39  | 465 | 27 | 107 | 16 | 1735 |
| 18:00:00   | 18:15:00 | 0  | 5  | 17 | 8  | 34  | 415 | 30 | 99  | 19 | 1782 |
| 18:15:00   | 18:30:00 | 0  | 3  | 28 | 10 | 37  | 430 | 39 | 108 | 20 | 1698 |
| 18:30:00   | 18:45:00 | 0  | 8  | 18 | 8  | 36  | 372 | 23 | 139 | 14 | 1462 |
| 18:45:00   | 19:00:00 | 0  | 5  | 13 | 8  | 32  | 363 | 35 | 105 | 16 | 1327 |
| 19:00:00   | 19:15:00 | 2  | 6  | 25 | 9  | 29  | 331 | 30 | 100 | 12 | 1347 |
| 19:15:00   | 19:30:00 | 1  | 10 | 13 | 4  | 26  | 348 | 31 | 88  | 8  | 1368 |
| 19:30:00   | 19:45:00 | 1  | 4  | 13 | 2  | 30  | 353 | 33 | 90  | 4  | 1309 |
| 19:45:00   | 20:00:00 | 1  | 5  | 5  | 0  | 20  | 339 | 30 | 82  | 5  | 1152 |
| 20:00:00   | 20:15:00 | 1  | 9  | 8  | 2  | 15  | 338 | 25 | 75  | 1  | 1077 |
| 20:15:00   | 20:30:00 | 2  | 13 | 4  | 0  | 14  | 321 | 33 | 64  | 2  | 887  |
| 20:30:00   | 20:45:00 | 4  | 7  | 3  | 0  | 11  | 259 | 30 | 71  | 1  | 668  |
| 20:45:00   | 21:00:00 | 1  | 11 | 2  | 0  | 7   | 268 | 26 | 59  | 0  | 587  |
| 21:00:00   | 21:15:00 | 7  | 8  | 0  | 1  | 4   | 231 | 19 | 53  | 0  | 525  |
| 21:15:00   | 21:30:00 | 16 | 8  | 0  | 3  | 3   | 227 | 15 | 58  | 1  | 475  |
| 21:30:00   | 21:45:00 | 19 | 5  | 0  | 0  | 4   | 228 | 15 | 43  | 0  | 398  |
| 21:45:00   | 22:00:00 | 21 | 5  | 0  | 1  | 3   | 171 | 17 | 33  | 0  | 335  |
| 22:00:00   | 22:15:00 | 4  | 1  | 0  | 0  | 3   | 66  | 1  | 15  | 0  | 125  |

Table D.0.3. PCU OD Data: Days 1 &amp; 2

| Route               | Day 1       |             | Day 2       |             |
|---------------------|-------------|-------------|-------------|-------------|
|                     | 9:15–10:15  | 17:45–18:45 | 9:00–10:00  | 17:00–18:00 |
| kupondole_tri       | 1127        | 775         | 1235        | 977         |
| kupondolw_m         | 1098        | 771         | 1331        | 782         |
| kupondole_maternity | 178         | 94          | 147         | 100         |
| maternity_kupondole | 266         | 323         | 276         | 322         |
| m_maternity         | 104         | 184         | 93          | 161         |
| m_kupondole         | 827         | 936         | 809         | 904         |
| m_tri               | 473         | 491         | 530         | 479         |
| tri_m               | 906         | 642         | 994         | 697         |
| tri_maternity       | 109         | 85          | 107         | 117         |
| tri_kupondole       | 374         | 534         | 402         | 539         |
| <b>Total PCU</b>    | <b>5462</b> | <b>4835</b> | <b>5924</b> | <b>5078</b> |

Table D.0.4. PCU OD Data: Days 3 &amp; 4

| Route               | Day 3       |             | Day 4       |             |
|---------------------|-------------|-------------|-------------|-------------|
|                     | 9:30–10:30  | 17:00–18:00 | 10:00–11:00 | 17:45–18:45 |
| kupondole_tri       | 992         | 993         | 1118        | 915         |
| kupondolw_m         | 1213        | 786         | 1214        | 779         |
| kupondole_maternity | 151         | 68          | 159         | 87          |
| maternity_kupondole | 279         | 302         | 69          | 90          |
| m_maternity         | 126         | 167         | 108         | 171         |
| m_kupondole         | 908         | 977         | 848         | 939         |
| m_tri               | 412         | 378         | 471         | 449         |
| tri_m               | 853         | 555         | 918         | 631         |
| tri_maternity       | 109         | 79          | 108         | 94          |
| tri_kupondole       | 350         | 433         | 375         | 502         |
| <b>Total PCU</b>    | <b>5393</b> | <b>4738</b> | <b>5388</b> | <b>4657</b> |

Table D.0.5. PCU OD Data: Day 5

| Route               | Day 5       |             |
|---------------------|-------------|-------------|
|                     | 9:30–10:30  | 17:15–18:15 |
| kupondole_tri       | 789         | 803         |
| kupondolw_m         | 1162        | 959         |
| kupondole_maternity | 66          | 33          |
| maternity_kupondole | 150         | 168         |
| m_maternity         | 41          | 30          |
| m_kupondole         | 775         | 835         |
| m_tri               | 422         | 418         |
| tri_m               | 656         | 640         |
| tri_maternity       | 89          | 94          |
| tri_kupondole       | 395         | 437         |
| <b>Total PCU</b>    | <b>4545</b> | <b>4417</b> |

## REFERENCES

- [1] L. Maharjan, “Performance evaluation and improvement of an intersection - a case study of thapathali intersection,” Master’s thesis, I.O.E. Pulchowk Campus, Dec 2023, accessed: Dec. 25, 2025. [Online]. Available: <https://elibrary.tucl.edu.np/items/2307a008-e706-4a16-9850-07f4ae87d458>
- [2] H. Wei, G. Zheng, V. Gayah, and Z. Li, “A survey on traffic signal control methods,” *arXiv preprint*, 2020, arXiv preprint. [Online]. Available: <https://arxiv.org/abs/2004.08117>
- [3] H. Wei, N. Xu, H. Zhang, G. Zheng, X. Zang, C. Chen, W. Zhang, Y. Zhu, K. Xu, and Z. Li, “Colight: Learning network-level cooperation for traffic signal control,” *arXiv preprint*, 2019, arXiv preprint. [Online]. Available: <https://arxiv.org/abs/1905.05717>
- [4] H. Wei, C. Chen, G. Zheng, K. Wu, V. Gayah, K. Xu, and Z. Li, “Presslight: Learning max pressure control to coordinate traffic signals in arterial network,” in *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*, 2019.
- [5] G. Zheng, Y. Xiong, X. Zang, J. Feng, H. Wei, H. Zhang, Y. Li, K. Xu, and Z. Li, “Learning phase competition for traffic signal control,” 2019, available online. [Online]. Available: <https://arxiv.org/abs/1912.09111>
- [6] S. Team, *Simulation of Urban MObility (SUMO)*, <https://www.eclipse.org/sumo/>, 2025, accessed: 2025-12-15. [Online]. Available: <https://www.eclipse.org/sumo/>